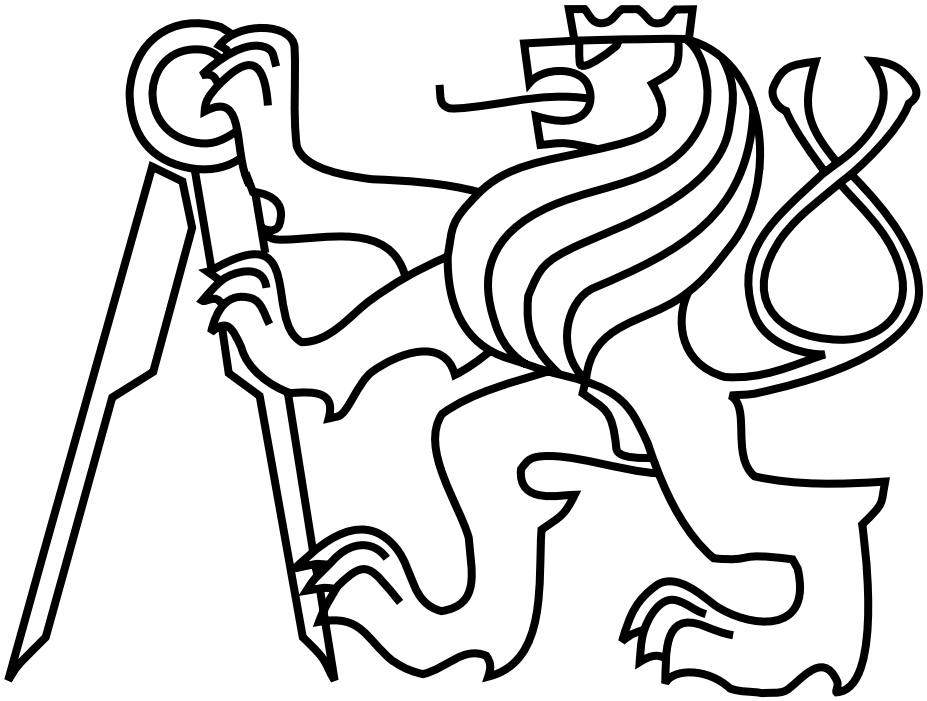
Czech Technical University in Prague
Faculty of Electrical Engineering
Department of Computer Science and Engineering



MASTER THESIS

Optimisation of Rectangular Shapes Placement by Means of Evolutionary Algorithms

Bc. Vojtěch Hordějčuk

Supervisor: Ing. Jiří Kubalík, Ph.D.

Study Programme: Elektrotechnika a informatika, strukturovaný, magisterský

Field of Study: Výpočetní technika, Softwarové inženýrství

Prague, 2011
Revised edition, 2026

Acknowledgements

My supervisor, Mr. Ing. Jiří Kubalík Ph.D., for valuable advice and research insight.

Statement

I hereby declare that I have completed this thesis independently and that I have listed all the literature and publications used within the research and work. I have no objections to usage of this work in compliance with Act No 121/2000 Sb. (the Copyright Act), as amended, and in compliance with copyright-related rights currently in force.

Prague, 2011 .....

Note on the revised edition. The statement above refers to the original edition of 2011, which is preserved in the history of the project repository. This revised edition (2026) was prepared at the author's request with the help of the AI assistant Claude (Anthropic). The revision consists of corrections of defects found in the original implementation, a faster implementation, improvements of the original algorithm, a new algorithm (late acceptance POEMS, Chapter 3), all experiments of Chapter 5, which were re-run from scratch with a new methodology, and a rewritten text. The results of the original edition are cited where they are compared.

Abstract

The thesis studies the placement of rotatable rectangles in the plane so that the enclosing rectangle is as small as possible, a model of VLSI floorplanning. Floorplans are represented by B*-Trees and optimised by POEMS, an iterative algorithm in which a genetic algorithm evolves short sequences of tree modifications (actions) that improve the current solution (the prototype). A critical re-examination of the original implementation found that most of its running time was spent on avoidable overhead, that several defects changed the behaviour of the algorithm (for example, modules could never be rotated alone), and that the genetic algorithm spent most of its evaluations on duplicates and on generations after the best step of the iteration had already been found. An amortised linear placement and an array-based B*-Tree with in-place actions make the evaluation 7.5 to 29 times faster without changing the results. Building on these findings, the thesis proposes *late acceptance POEMS* (LA-POEMS), which applies random POEMS action sequences immediately and accepts them by simulated annealing or late acceptance. In experiments with equal time budgets, new random seeds, held-out instances and non-parametric tests on the MCNC and GSRC benchmarks, LA-POEMS reduces the dead space of the original algorithm by 57–73 % within one minute on instances with 33 to 300 modules and is significantly better than an improved POEMS on most instances. Its results are as good as or better than those of the original edition after up to 2.5 hours of computation and the published results of the CompaSS packer.

Abstrakt

Práce se zabývá rozmístěním otočitelných obdélníků v rovině tak, aby obdélník, který je ohraničuje, byl co nejmenší, což je model rozmisťování bloků na čipu (floorplanning). Rozmístění je reprezentováno stromem B*-Tree a optimalizováno algoritmem POEMS, iterativním algoritmem, ve kterém genetický algoritmus hledá krátké posloupnosti úprav stromu (akcí), jež zlepšují aktuální řešení (prototyp). Kritický rozbor původní implementace ukázal, že většinu času běhu spotřebovala zbytečná režie, že několik chyb měnilo chování algoritmu (například moduly nebylo možné samostatně otočit) a že genetický algoritmus většinu vyhodnocení věnoval duplicitním jedincům a generacím, ve kterých už byl nejlepší krok iterace nalezen. Rozmisťování v amortizovaně lineárním čase a reprezentace stromu v polích s akcemi prováděnými na místě zrychlují vyhodnocení 7,5krát až 29krát, aniž by se změnily výsledky. Na základě těchto zjištění práce navrhuje *POEMS s pozdním přijetím* (late acceptance POEMS, LA-POEMS), který náhodné posloupnosti akcí aplikuje okamžitě a přijímá je pomocí simulovaného žíhání nebo pozdního přijetí. V experimentech se stejným časovým limitem, novými semínky náhodného generátoru, úlohami nepoužitými při ladění a neparametrickými testy na testovacích úlohách MCNC a GSRC snižuje LA-POEMS během jedné minuty nevyužitou plochu původního algoritmu o 57 až 73 % na úlohách s 33 až 300 moduly a na většině úloh je významně lepší než vylepšený POEMS. Jeho výsledky jsou stejně dobré nebo lepší než výsledky původní verze práce po až 2,5 hodinách výpočtu i publikované výsledky nástroje CompaSS.

Contents

1 Introduction **1**

1.1 The Approach of this Thesis 1

1.2 Contributions 2

1.3 Research Questions 2

1.4 Summary of Results 3

1.5 Thesis Structure 3

2 Floorplanning: Problem and Related Work **5**

2.1 Problem Formulation 5

2.2 Complexity 6

2.3 Floorplan Representations 6

2.3.1 Slicing Trees 7

2.3.2 Sequence Pairs 7

2.3.3 O-Trees and B*-Trees 9

2.4 Optimisation Methods 11

2.4.1 Simulated Annealing 11

2.4.2 Evolutionary Algorithms 11

2.4.3 POEMS 11

2.5 Benchmarks 12

3 Methods **13**

3.1 Placement of a B*-Tree . . . 13

3.1.1 The Contour . . . 13

3.1.2 Early Termination . . . 14

3.2 The Prototype . . . 14

3.3 POEMS for B*-Trees . . . 15

3.3.1 Actions . . . 15

3.3.2 Action Sequences and Niches . . . 15

3.3.3 The Genetic Algorithm . . . 16

3.4 What the Genetic Algorithm Does . . . 17

3.5 Improvements of POEMS . . . 18

3.6 Late Acceptance POEMS . . . 19

3.6.1 Acceptance . . . 19

3.6.2 Further Options . . . 20

3.6.3 Relation to POEMS . . . 20

4 Implementation **21**

4.1 Architecture . . . 21

4.2 The Original Design and its Cost . . . 21

4.3 Speed-ups that Do Not Change the Results . . . 22

4.4 The Array B*-Tree . . . 23

4.5 Defects Found and Fixed . . . 24

4.6 Correctness and Reproducibility . . . 24

5 Experimental Evaluation **27**

5.1 Methodology . . . 27

5.2 RQ1: The Cost of the Implementation . . . 28

5.3 RQ2: Corrections and Improvements of POEMS . . . 31

5.3.1 Effect of the Corrections . . . 31

5.3.2 Improvements of the Genetic Algorithm . . . 32

5.4 RQ3: Design of LA-POEMS . . . 33

5.4.1 Single Actions . . . 34

5.4.2 Acceptance . . . 34

5.4.3 Steps and Other Options . . . 35

5.4.4 The Final Configuration . . . 37

5.4.5 LA-POEMS and POEMS at Equal Speed . . . 37

5.5 RQ4: Final Comparison . . . 38

5.5.1 Tuning Benchmarks . . . 38

5.5.2 Held-Out Benchmarks . . . 39

5.5.3 Convergence . . . 39

5.5.4 Long Runs and Published Results . . . 41

5.6 Threats to Validity . . . 43

6 Conclusions **45**

6.1 Summary . . . 45

6.2 Lessons . . . 46

6.3 Future Work . . . 46

A Using the Program **49**

A.1 Building . . . 49

A.2 Command Line . . . 49

A.3 Benchmark Files . . . 51

A.4 Reproducing the Experiments . . . 51

List of Figures

2.1 An example instance and a feasible placement of its modules 6

2.2 Slicing floorplans and their slicing trees 8

2.3 A floorplan encoded by the sequence pair (ecadfb, fcbead) 8

2.4 A floorplan encoded by the O-Tree (001101000111, fbcead) 9

2.5 A floorplan encoded by a B*-Tree 10

2.6 The contour of the placed modules and its use for placing a new module . . 10

3.1 The two constructions of the prototype 15

3.2 POEMS for B*-Trees 17

3.3 Late acceptance POEMS (with the two acceptance criteria) 19

4.1 Packages and the most important classes (arrows show dependencies) . . . 22

5.1 Time of one placement of a complete B*-Tree with random module sizes: the original contour, the array contour with the immutable tree, and the array B*-Tree (median of 9 batches after a warm-up; logarithmic axes) . . . 29

5.2 Convergence of the three algorithms on ami49, n100 and n300: median dead space of the best floorplan found so far over the 8 runs (seeds 101–108), with the interquartile range shaded; logarithmic time axis 40

5.3 The best floorplans found by LA-POEMS in the final comparison and in the long runs, with the dead space in gray (the longer side is drawn horizontally). The objective does not constrain the aspect ratio, so some floorplans, such as that of apte, are long strips 42

A.1 The MCNC hp benchmark file 51

List of Tables

2.1 Selected floorplan representations (sizes of the solution spaces as given in [10]) 7

2.2 Benchmarks (areas in the units of the benchmark files) 12

3.1 The actions modifying a B*-Tree 16

3.2 Behaviour of the genetic algorithm of POEMS ($G = 1000$, $N = 3$, $S = 50$, 20 s runs, seed 1). Children: duplicates of an individual of the population (same enabled actions), children that do not change the prototype, and children with the same fitness as a parent. Iterations: the best sequence of the iteration improves the prototype, has the same fitness, or is worse. Best found in generation: mean / median generation of the last improvement of the best individual of an iteration. 18

4.1 Profile of the original implementation (n100, sampling profiler) 22

4.2 Defects of the original implementation 24

5.1 Evaluations per second (thousands) of the original implementation of POEMS, the optimised implementation, the corrected one, the corrected one with the array evaluation, and LA-POEMS, whose evaluations stop early when the candidate is rejected (mean of the 30 s runs; speed-up relative to the original implementation in parentheses) 30

5.2 Dead space [%] of the original implementation and of the implementation optimised without changing the results (same algorithm and seeds, 30 s, 8 seeds) 30

5.3 Dead space [%] before and after the corrections of Section 4.5 (30 s, 8 seeds) 31

5.4 Outcome of the iterations of POEMS with the original identity check, which ignores the orientation of the modules, and with the corrected one (20 s, seed 1) 32

5.5 Modifications of the genetic algorithm of POEMS, each applied alone to the reference configuration (array evaluation, $G = 1000$, $N = 3$, $S = 50$): fewer generations per iteration, discarding duplicate children, the move node action, ending an iteration after 100 generations without improvement or at the first improvement of the prototype, and late acceptance of prototypes with a history of 20 iterations (dead space [%], 30 s, seeds 1–6) 33

5.6 Outcome of the steps of LA-POEMS that consist of a single action, by action type (late acceptance with $L = 1000$, steps of at most three actions of uniformly chosen types, 10 s, seed 1; about 10^5–$5 \cdot 10^5$ single-action steps per type) 34

5.7 Acceptance criteria of LA-POEMS: late acceptance with a fixed history length L or with the automatic length (ratio C), and simulated annealing with the initial temperature T_0 relative to the area of the modules (final temperature 10^{-6}, or 10^{-5} for $T_0 = 10^{-2}$); steps of at most three actions (dead space [%], 30 s, seeds 1–6) 35

5.8 Options of LA-POEMS with late acceptance ($L = 1000$): steps of exactly one action and of at most two actions instead of three, weights of the action types favouring flip, rotate, exchange value and move node (3:2:1:3:1:1:2), restarts after $5 \cdot 10^4 n$ steps without improvement, and local partners with the probability 0.5 and 0.25 (dead space [%], 30 s, seeds 1–6) 36

5.9 Options of LA-POEMS with simulated annealing ($T_0 = 10^{-2}$, $T_1 = 10^{-5}$, steps of at most three actions): steps of exactly one and of at most two actions, local partners with the probability 0.5 (alone and with steps of at most two actions), adaptive selection of the action types, and, for comparison, late acceptance with the automatic history ($C = 4 \cdot 10^4$) and restarts (dead space [%], 30 s, seeds 1–6) 36

5.10 POEMS and LA-POEMS on the same array implementation: POEMS with the parameters of the original edition, the first design of LA-POEMS (late acceptance, $L = 1000$, steps of at most three actions) and the final configuration of LA-POEMS; both LA-POEMS columns are compared with POEMS (dead space [%], 30 s, seeds 1–6) 37

5.11 Final comparison of the original POEMS, the improved POEMS and LA-POEMS on the tuning benchmarks (dead space [%], 60 s, seeds 101–108; mean, standard deviation and best run). Each algorithm is tested against the column to its left; the last columns give the probability that a run of LA-POEMS ends with less dead space than a run of the other algorithm . . 39

5.12 Final comparison of the original POEMS, the improved POEMS and LA-POEMS on the held-out benchmarks (dead space [%], 60 s, seeds 101–105; mean, standard deviation and best run). Each algorithm is tested against the column to its left; the last columns give the probability that a run of LA-POEMS ends with less dead space than a run of the other algorithm . . 40

5.13 Median time [s] in which the improved POEMS and LA-POEMS first reach the median final dead space of the original POEMS (the target, 60 s runs, seeds 101–108; – = not reached by half of the runs) 41

5.14 LA-POEMS compared with the original edition and with published results: dead space [%] of LA-POEMS (mean and best of 8 runs of 60 s, seeds 101–108, and of 5 runs of 300 s, seeds 101–105), the mean of 10 runs of the longest configuration of the original edition ($I = 20\,000$, on a 2 GHz computer), CompaSS [27, 28] (2.8 GHz Pentium 4) and the B*-Tree with simulated annealing [4] (computed from the areas reported with two decimals, 200 MHz Sun Ultra-I); times as reported 42

A.1 Mandatory command line arguments 50

A.2 Optional command line arguments 50

A.3 System properties of the experimental settings 52

Chapter 1

Introduction

Floorplanning is an early step of the physical design of integrated circuits: the functional blocks (modules) of a chip are placed on the plane so that they do not overlap and so that the chip is as small as possible, possibly subject to further objectives such as the wire length. In its simplest form, which is studied in this thesis, the modules are rectangles of fixed dimensions that may be rotated by 90 degrees, and the only objective is the area of the rectangle enclosing all of them. The same rectangle packing model appears in other domains, for example in scheduling, where the width of a rectangle is the duration of a task and its height the amount of a resource used by it. The decision version of the problem is NP-complete, so practical floorplanners rely on heuristics and metaheuristics.

1.1 The Approach of this Thesis

The thesis combines two ideas. Floorplans are represented by *B*-Trees* [4], ordered binary trees from which a compact placement is computed in linear time. The trees are optimised by *POEMS* (Iterative Prototype Optimisation with Evolved Improvement Steps) [15]: an initial solution, the prototype, is improved in iterations, and in each iteration a genetic algorithm evolves a short sequence of modifications of the prototype (*actions*) whose application gives the best solution. The best sequence found in an iteration is applied to the prototype if the result is not worse.

The original edition of the thesis (2011) designed the actions for B*-Trees, implemented the algorithm and reported results on standard benchmarks. This revised edition re-examines that work critically: it profiles and tests the implementation, analyses what the genetic algorithm actually does, corrects the defects it found, improves the original algorithm and proposes a new algorithm derived from it. All experiments were run again with a new methodology: equal time budgets, fixed and independent random seeds, independent validation of every result and non-parametric statistical tests.

1.2 Contributions

1. **POEMS for B*-Tree floorplanning** (original edition): a set of actions that modify B*-Trees, action sequences organised into niches by the number of enabled actions, and a steady-state genetic algorithm that evolves them (Chapter 3). The revised edition adds a seventh action, the move of a single node.

2. **A critical analysis of the original implementation**: profiling (most of the time was spent outside the algorithm), seven defects that changed the behaviour of the algorithm or made parts of it unusable (for example, rotation alone was never applied and a sequence length of one made the program hang), and instrumentation of the genetic algorithm showing that most of its evaluations are wasted (Chapters 3 and 4).

3. **An efficient implementation**: an amortised linear contour placement, lazy construction of placed trees and an array-based B*-Tree with in-place actions that are tested to produce exactly the same trees as the original ones. POEMS evaluates 7.5 to 29 times more candidate floorplans per second with identical results; the speed-up grows with the number of modules (Chapters 4 and 5).

4. **Modifications of the genetic algorithm of POEMS**, each evaluated separately: fewer generations per iteration, duplicate elimination, early termination of an iteration, late acceptance of new prototypes and the new action; only shorter iterations improve POEMS (Chapter 5).

5. **Late acceptance POEMS (LA-POEMS)**, a new algorithm that keeps the representation, the actions and the action sequences of POEMS, but applies random sequences to the prototype immediately and accepts the results by late acceptance hill climbing [2] or simulated annealing [14], together with an ablation of its design choices (Chapters 3 and 5).

6. **A rigorous experimental comparison** of the original algorithm, the improved POEMS and LA-POEMS with equal time budgets on the tuning benchmarks, on held-out benchmarks and in long runs, and a comparison with the results of the original edition and with the literature (Chapter 5).

1.3 Research Questions

RQ1 How much of the running time of the original implementation is overhead that does not change the computed results, and how does removing it change the quality of solutions found within a given time?

RQ2 How efficiently does the genetic algorithm of POEMS use its evaluations, and which modifications of POEMS improve its results?

RQ3 Is evolving the improvement steps by a genetic algorithm necessary, or is it better to apply random steps immediately with a suitable acceptance criterion (LA-POEMS)? Which design choices of LA-POEMS matter?

RQ4 How do the original algorithm, the improved POEMS and LA-POEMS compare with equal time budgets, on held-out instances, in long runs, and with the results of the original edition and of the literature?

1.4 Summary of Results

RQ1 Most of the running time of the original implementation was overhead. Removing it makes POEMS 4 to 8 times faster with identical results, and the array evaluation 7.5 to 29 times; within the same 30 seconds, the faster implementation alone finds significantly better floorplans on all five benchmarks with more than 30 modules.

RQ2 Almost half of the children of the genetic algorithm duplicate individuals of the population, the step applied in an iteration is usually present in its initial random population, and at most 3.4 % of the iterations improve the prototype. Shorter iterations (fewer generations) make POEMS significantly better on four of seven benchmarks; the other modifications do not help.

RQ3 With the same actions, implementation and time, random steps accepted by simulated annealing are significantly better than the evolved steps of POEMS on most instances and worse on none. The acceptance criterion matters most; short steps and local partners help on large instances.

RQ4 With equal time budgets of one minute and new seeds, LA-POEMS reduces the dead space of the original POEMS by 57–73 % on the instances with 33 to 300 modules and is significantly better than the improved POEMS on most tuning and held-out instances. Its results are as good as or better than those of the original edition after up to 2.5 hours and the published results of CompaSS, and its best floorplans are at least as good as those of the B*-Tree with simulated annealing; simulated annealing, however, needs to know the time budget in advance.

1.5 Thesis Structure

Chapter 2 formulates the problem, reviews floorplan representations and the optimisation methods used for floorplanning. Chapter 3 describes the B*-Tree placement, POEMS with

its actions and genetic algorithm, the analysis of the genetic algorithm, the improvements of POEMS and the new LA-POEMS algorithm. Chapter 4 describes the implementation, its performance engineering, the defects found in the original implementation and the measures taken to ensure correctness and reproducibility. Chapter 5 presents the experimental methodology and answers the research questions, and Chapter 6 concludes the thesis. Appendix A describes the use of the program and how to reproduce the experiments.

Chapter **2**

Floorplanning: Problem and Related Work

This chapter defines the problem, summarises its complexity, reviews the representations of floorplans used by optimisation algorithms and the optimisation methods applied to floorplanning, and describes the benchmarks used in this thesis.

2.1 Problem Formulation

An instance consists of n modules $M = \{1, \ldots, n\}$; module i is a rectangle of width w_i and height h_i (positive integers). A *placement* assigns to every module a position (x_i, y_i) of its lower left corner and an orientation $r_i \in \{0, 1\}$: a rotated module ($r_i = 1$) has its dimensions swapped. Let (w'_i, h'_i) denote the dimensions after rotation. A placement is *feasible* if no two modules overlap:

$$\forall i \neq j: \quad x_i + w'_i \leq x_j \ \vee \ x_j + w'_j \leq x_i \ \vee \ y_i + h'_i \leq y_j \ \vee \ y_j + h'_j \leq y_i. \tag{2.1}$$

With all coordinates non-negative, the enclosing rectangle has the width $W = \max_i(x_i + w'_i)$ and the height $H = \max_i(y_i + h'_i)$, and the task is to find a feasible placement with the minimal area $A = W \cdot H$. The quality of a placement is reported as the *dead space*

$$d = \frac{W \cdot H}{\sum_{i \in M} w_i h_i} - 1, \tag{2.2}$$

the unused area relative to the area of the modules, which is the measure used in the original edition and in much of the floorplanning literature. A placement with $d = 0$ uses

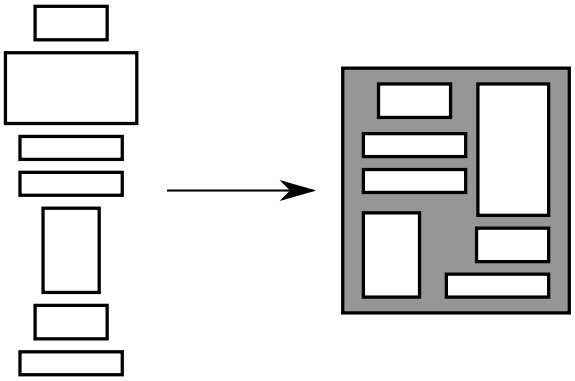


Figure 2.1: An example instance and a feasible placement of its modules

the enclosing rectangle perfectly; the optimal dead space of a given instance is usually unknown and may be greater than zero. Fig. 2.1 shows an example.

In practical floorplanning, the area is usually combined with the total wire length of the nets connecting the modules, and modern floorplanners often place the modules into a fixed outline given in advance instead of minimising the area [1]. The thesis studies the classical area minimisation of hard (fixed-shape) rotatable modules, which is the setting of the original edition and of many papers on floorplan representations [4, 10, 21].

2.2 Complexity

The decision version of the problem, whether the modules fit into a rectangle of given width and height, is NP-complete in the strong sense: if all modules have height one and widths greater than k, so that none of them can be rotated, and the rectangle has height k, the modules have to be packed into k rows, which is exactly the strongly NP-complete bin packing problem with k bins [8]. The problem remains strongly NP-complete when all modules are squares [17], and related packing problems in the plane were shown to be NP-complete already by Fowler et al. [7]. Consequently, exact methods are limited to small instances and floorplanners rely on heuristics and metaheuristics, which search over an encoding of the floorplans, the *representation*.

2.3 Floorplan Representations

A representation encodes a set of floorplans by combinatorial objects. Good representations can encode an optimal floorplan, can be decoded quickly into a placement, and allow simple

Representation	Floorplans	Size of the solution space	Decoding
Slicing tree / Polish expression [25]	slicing	$O(n!\,2^{5n-3}/n^{1.5})$	$O(n)$
Sequence pair [21]	general	$O((n!)^2)$	$O(n^2)$
O-Tree [10]	admissible	$O(n!\,2^{2n-2}/n^{1.5})$	$O(n)$
B*-Tree [4]	admissible	$O(n!\,2^{2n-2}/n^{1.5})$	$O(n)$

Table 2.1: Selected floorplan representations (sizes of the solution spaces as given in [10])

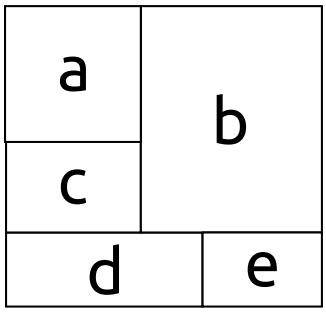
modifications (moves) for local search or evolutionary algorithms. Floorplans are divided into *slicing* floorplans, which can be obtained by recursively cutting the enclosing rectangle into two by horizontal or vertical lines, and general *non-slicing* floorplans. Table 2.1 summarises the representations discussed below.

2.3.1 Slicing Trees

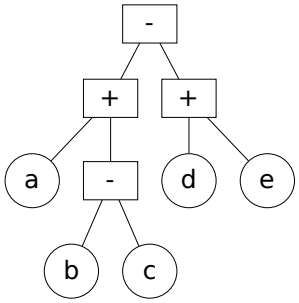
A slicing floorplan is represented by a binary tree whose leaves are the modules and whose internal nodes are the operators of a vertical or a horizontal join of the two sub-floorplans. Written in post-order, the tree becomes a Polish expression, the representation of the classical simulated annealing floorplanner of Wong and Liu [25]. Slicing floorplans are easy to decode, but they cannot represent every floorplan directly; Lai and Wong [16] showed that, combined with compaction, slicing trees can generate all maximally compact placements. Fig. 2.2 shows two examples.

2.3.2 Sequence Pairs

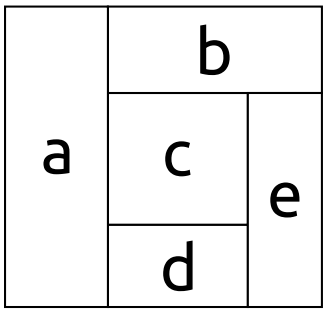
A sequence pair [21] is a pair of permutations of the modules. The relative order of two modules in both sequences determines their geometric relation: if a precedes b in both sequences, a lies to the left of b; if a precedes b in the first sequence and follows it in the second, a lies above b. Every sequence pair corresponds to a feasible placement, which is obtained by computing the longest paths in the horizontal and vertical constraint graphs. The representation can encode every floorplan, so its solution space always contains an optimal one. An example is given in Fig. 2.3. Related graph-based representations include the bounded slicing grid [22], the corner block list [12] and the transitive closure graph [19, 18].



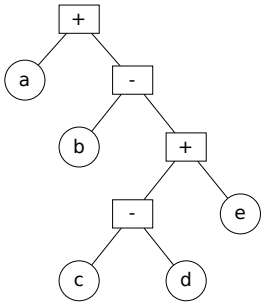
(a) floorplan 1



(b) slicing tree 1

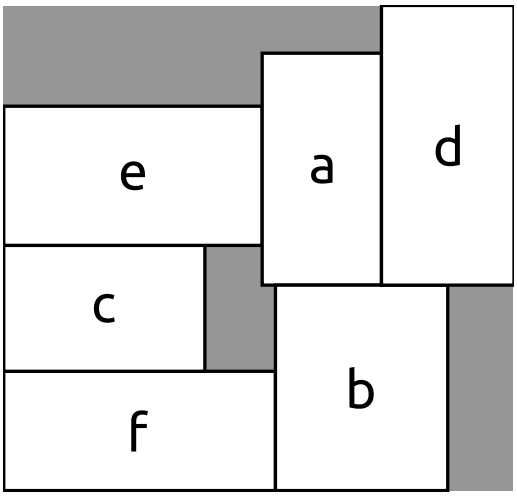


(c) floorplan 2

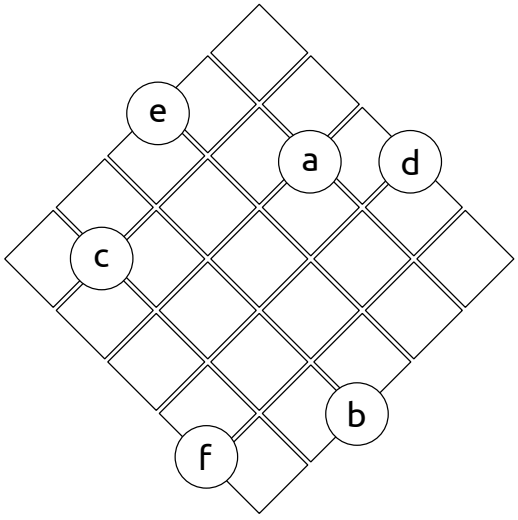


(d) slicing tree 2

Figure 2.2: Slicing floorplans and their slicing trees

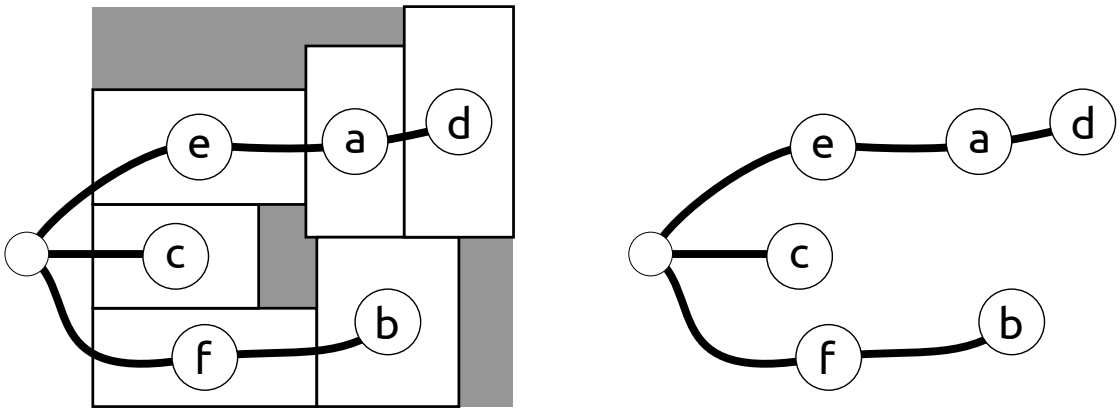


(a) floorplan



(b) sequence pair constraints

Figure 2.3: A floorplan encoded by the sequence pair (ecadfb, fcbead)



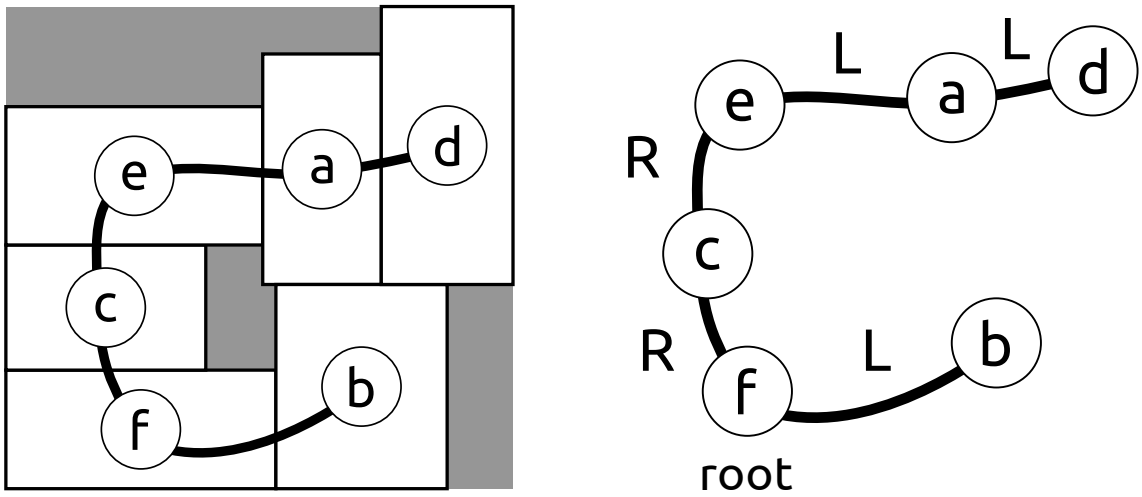
(a) floorplan **(b)** O-Tree

Figure 2.4: A floorplan encoded by the O-Tree (001101000111, fbcead)

2.3.3 O-Trees and B*-Trees

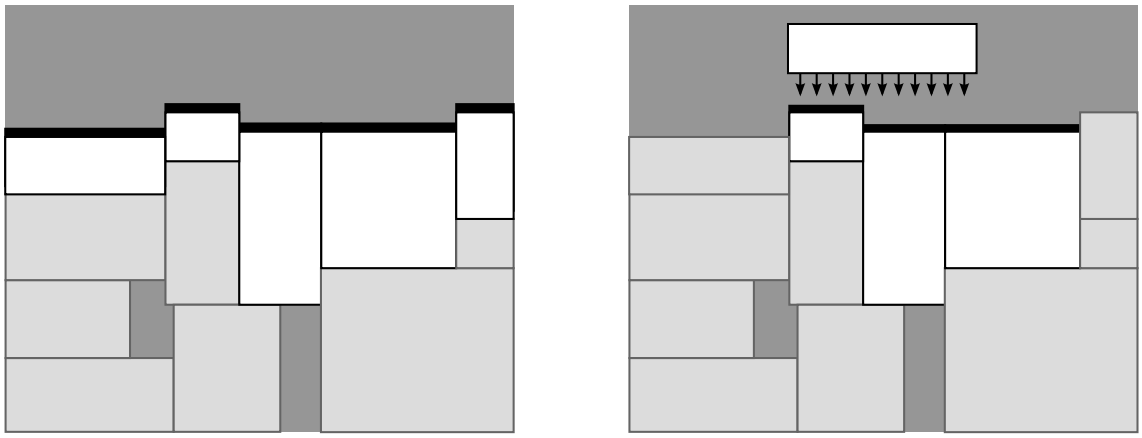
A placement is *L-compact* (*B-compact*) if no module can be moved to the left (down) without overlapping another module, and *admissible* if it is both. Every placement can be transformed into an admissible placement that is not larger by alternating compactions to the left and down, so it suffices to search among admissible placements. The O-Tree [10] represents admissible placements by ordered trees: a module is placed next to its parent and as low as the already placed modules allow. The B*-Tree [4] encodes the same information by an ordered binary tree. The root module is placed at the origin; the left child of a module is placed immediately to its right, and the right child immediately above it at the same x coordinate. The y coordinate of every module is the lowest one at which it does not overlap the modules placed before it in the depth-first (pre-order) traversal of the tree. Fig. 2.4 and Fig. 2.5 give examples.

The placement of a B*-Tree is computed with a *contour*, the upper envelope of the modules placed so far, kept as a list of horizontal segments. The y coordinate of a new module is the maximum height of the contour over its horizontal extent, after which the contour is raised to its top (Fig. 2.6). With the contour, the placement of the whole tree takes linear time [10, 4] (Section 3.1). The B*-Tree supports simple moves (rotation of a module, swap of two modules, removal and insertion of a node) and extensions to pre-placed, soft and rectilinear modules [4, 5], which made it popular in academic floorplanners and the representation of choice in this thesis.



(a) floorplan **(b)** B*-Tree

Figure 2.5: A floorplan encoded by a B*-Tree



(a) whole contour **(b)** contour part evaluated

Figure 2.6: The contour of the placed modules and its use for placing a new module

2.4 Optimisation Methods

2.4.1 Simulated Annealing

Simulated annealing [14] is the most common optimisation method in floorplanning. Starting from a random solution, it repeatedly applies a random move and accepts the result if it is not worse or, otherwise, with the probability $e^{-\Delta/T}$, where Δ is the deterioration and the temperature T decreases during the run. It was used with Polish expressions [25], sequence pairs [21], O-Trees [10] and B*-Trees [4]; the *Fast-SA* of Chen and Chang [5] uses a three-stage temperature schedule and B*-Trees and also addresses fixed-outline floorplanning. Late acceptance hill climbing [2] is a more recent alternative with a single parameter: a candidate is accepted if it is not worse than the current solution or than the solution L steps ago.

2.4.2 Evolutionary Algorithms

Evolutionary algorithms [11, 6] maintain a population of candidate solutions that are selected according to their quality (fitness) and recombined and mutated to create new candidates. Their application to floorplanning is complicated by the representations: a crossover of two trees or permutations usually needs to be repaired or specially designed. Memetic algorithms combine the evolutionary search with local search; Tang and Yao [23] report optimal or near optimal results on popular benchmarks with a hybrid of a genetic algorithm and local search for non-slicing floorplans of hard modules.

2.4.3 POEMS

POEMS [15] avoids recombining whole solutions. It works with a single solution, the *prototype*, which is improved in iterations. In each iteration, an evolutionary algorithm searches the space of short sequences of *actions*, parameterised modifications of the prototype; the fitness of a sequence is the quality of the prototype modified by it. The best sequence found in the iteration is applied to the prototype if the result is not worse. POEMS was introduced on binary string optimisation and the travelling salesman problem [15]. The author's bachelor thesis [13] applied it to floorplanning with slicing trees and the original edition of this thesis to B*-Trees.

Benchmark	Modules	Area of modules	Held-out	Modules	Area of modules
apte	9	46 561 628	n30b	30	197 781
xerox	10	19 350 296	n50b	50	203 053
hp	11	8 830 584	n100b	100	160 126
ami33	33	1 156 449	n100c	100	171 966
ami49	49	35 445 424	n200b	200	174 593
n100	100	179 501	n200c	200	170 129
n200	200	175 696			
n300	300	273 170			

Table 2.2: Benchmarks (areas in the units of the benchmark files)

2.5 Benchmarks

The experiments use the MCNC benchmarks apte, xerox, hp, ami33 and ami49 and the GSRC benchmarks n100, n200 and n300 in the hard-block form distributed with the floorplanning tools of the University of Michigan [26]: a text file with the number of modules followed by their dimensions. The GSRC variants n30b, n50b, n100b, n100c, n200b and n200c, which were not used during the development and tuning of the algorithms, serve as held-out instances. Table 2.2 lists their sizes. Only the dimensions of the modules are used; the connectivity information of the original benchmarks is ignored, as the objective is the area.

Chapter 3

Methods

This chapter describes how a B*-Tree is turned into a placement, how the initial solution is constructed, the POEMS algorithm for B*-Trees from the original edition, an analysis of the behaviour of its genetic algorithm, the improvements of POEMS evaluated in Chapter 5, and the new late acceptance POEMS.

3.1 Placement of a B*-Tree

The modules are placed in the depth-first pre-order of the tree (a node, then its left subtree, then its right subtree). The root is placed at $x = 0$; the left child of a node p at $x = x_p + w'_p$ and the right child at $x = x_p$. The y coordinate of a module is the lowest one at which it does not overlap any module placed before it, that is, the maximum of $y_j + h'_j$ over the placed modules j whose horizontal extent overlaps the interval $[x, x + w')$, or zero if there is none.

3.1.1 The Contour

The *contour* is the upper envelope of the placed modules: a list of horizontal segments $[a_k, b_k)$ ordered from the left, covering $[0, \infty)$, each with the height (level) of the highest module above it. To place a module at x, the segments overlapping $[x, x + w')$ are walked and the maximal level is its y coordinate; then the walked segments are replaced by a single segment $[x, x + w')$ at the level $y + h'$, keeping the parts of the first and the last segment that are not covered. Since the new level is above every replaced level, the list

stays the upper envelope, and the maximal level over an interval is the maximal top of the modules overlapping it.

In the pre-order traversal, the segment where a module starts is known without searching. The left child of p starts at $x_p + w'_p$, which is the start of the segment following the segment of p; the right child starts at x_p, the start of the segment of p. The segment of p is not changed before its right child is placed, because all modules of the left subtree of p lie to the right of $x_p + w'_p$. Hence no search is needed, every placement creates at most one new segment (the uncovered rest of the last walked segment), and every walked segment except the last one is removed. The placement of the whole tree therefore takes $O(n)$ time, as stated in [10, 4].

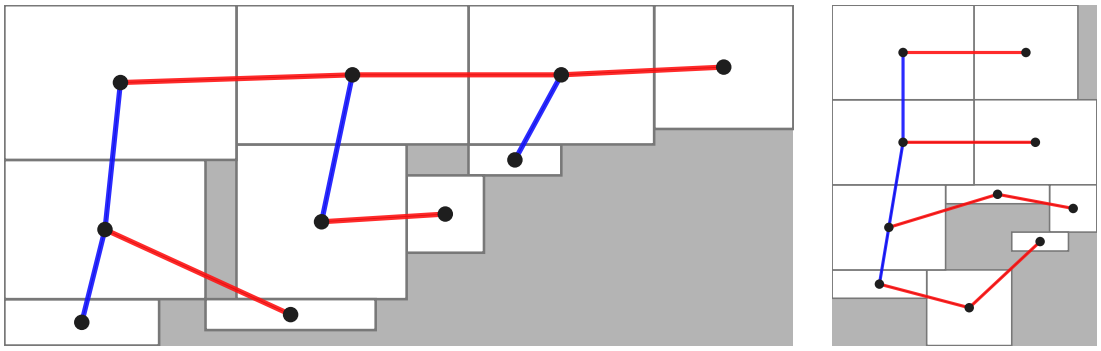
The original implementation kept all placed modules in a priority queue ordered by their tops and walked it until the first module overlapping the new one horizontally. This gives the same coordinates but takes $O(n)$ time per module in the worst case and $O(n^2)$ per placement; Section 5.2 compares both.

3.1.2 Early Termination

During the placement, the width and the height of the enclosing rectangle of the modules placed so far never decrease. If a search algorithm only needs to know whether the area of a candidate is at most some threshold θ (for example, the area that would still be accepted), the placement can stop as soon as $W \cdot H > \theta$: the candidate would be rejected anyway. This does not change any decision of the algorithm, only saves time on the rejected candidates.

3.2 The Prototype

POEMS starts from an initial B*-Tree, the prototype. The original edition proposed two constructions; the *best-fit* construction was used in all experiments. First, every module is rotated to be at least as wide as high, and the modules are sorted by width (descending) and then by height (descending). The floorplan is then filled shelf by shelf. The width of a shelf is the larger of the widest module and the square root of the total area of the modules. The next module placed on the current shelf is the first module in the sorted order that fits into its remaining width. If no module fits, a new shelf is started. In the tree, a module placed next to the previous one becomes its left child, and the first module of a new shelf becomes the right child of the first module of the previous shelf. The alternative *complete*



(a) complete tree **(b)** best-fit heuristic

Figure 3.1: The two constructions of the prototype

tree construction stores the sorted modules in a complete binary tree in the order of an array-based heap. Fig. 3.1 shows both.

The original implementation of the best-fit construction iterated over a priority queue, which in Java does not iterate in the priority order, so it did not pick the widest fitting module as described. Corrected, it produces prototypes with somewhat less dead space on four of the eight benchmarks (for example 17.0 % instead of 18.5 % on n300 and 20.8 % instead of 22.7 % on xerox) and the same prototypes on the others.

3.3 POEMS for B*-Trees

3.3.1 Actions

The nodes of the prototype are numbered $0, \ldots, n-1$ in pre-order. A number identifies a module: when an action moves a module, its number moves with it. An action has an *enabled* flag and parameters (numbers of nodes and flags); a disabled action does nothing. Table 3.1 lists the seven actions. The first six come from the original edition; the *move node* action was added in this revision. It is the standard B*-Tree move of a single node [4], finer than moving whole subtrees by the hang node action.

3.3.2 Action Sequences and Niches

An individual of the genetic algorithm is a sequence of N actions applied to the prototype one after another. The number of its enabled actions is its *niche*. If the enabled and

Action	Effect
flip(a, r)	rotates the module of node a (if r, also all modules in its subtree)
rotate(a)	in the subtree of a, swaps the children of every node and rotates every module, which transposes the sub-floorplan
mirror(a, r)	swaps the children of node a (if r, of all nodes in its subtree)
exchange value(a, b)	swaps the modules of nodes a and b; the tree structure is kept
exchange node(a, b)	swaps the subtrees rooted at a and b if neither contains the other
hang node(a, b, s)	moves the subtree of a to become the left (s) or right child of b; the original child of b takes the place of a (nothing is done if b lies in the subtree of a or a in the subtree of b on side s)
move node(a, b, s)	removes node a (its left child takes its place and its right subtree is attached above the rightmost branch of the left child) and inserts it as the left (s) or right child of b, with the original child of b on the same side

Table 3.1: The actions modifying a B*-Tree

the disabled actions were treated equally, the population would soon consist of sequences with few enabled actions, as most modifications of a good prototype make it worse. The population is therefore divided into N niches of S individuals each; niche k contains only sequences with at least k enabled actions.

The fitness of a sequence is the negative unused area $-(W \cdot H - \sum_i w_i h_i)$ of the modified prototype. A sequence that does not change the prototype, for example two flips of the same module, gets the minimal fitness, so that the algorithm searches for real changes. The original implementation compared the trees by the node numbers only. A sequence that just rotated modules was therefore considered unchanged and rejected, so rotations were only ever applied together with structural changes. The comparison now includes the orientations of the modules.

3.3.3 The Genetic Algorithm

Each iteration runs a steady-state genetic algorithm for G generations on a new random population (Fig. 3.2). In a generation, a niche is chosen at random and parents are selected from it by a binary tournament in which the better individual wins with the probability 0.95 [9]. With the probability 0.7, two parents are recombined by the uniform crossover of their action sequences; otherwise one parent is mutated. The mutation either changes the order of the actions (probability 0.75) or replaces each action with the probability 0.4 by a variant of the same type with possibly changed flags and parameters. A new individual replaces the first individual in the population (scanning from the first niche to its own) that has a worse or equal fitness; if there is none, it is discarded. A sequence without

```
1: β ← the best-fit prototype; β* ← β
2: while time and iterations remain do
3:    create a population of N · S random action sequences (niche k: k enabled actions)
4:    for g = 1, ..., G do
5:       choose a niche; select parents from it by binary tournaments
6:       create one or two children by mutation or uniform crossover and evaluate them on
         β
7:       insert each child in place of the first individual that is not better
8:    end for
9:    σ ← the best sequence of the iteration
10:   if f(σ(β)) ≥ f(β) then
11:      β ← σ(β)
12:   end if
13:   if f(β) ≥ f(β*) then
14:      β* ← β
15:   end if
16: end while
17: return β*
```

Figure 3.2: POEMS for B*-Trees

enabled actions is replaced by a random one. The best sequence of the iteration is applied to the prototype if the result is not worse than the prototype. The original edition used $G = 1000$, $N = 3$ and $S = 50$.

3.4 What the Genetic Algorithm Does

To see how POEMS spends its evaluations, the genetic algorithm was instrumented. Every child was compared with its parents and with the individuals in the population, and for every iteration the generation of the last improvement of the best individual and the outcome of the iteration were recorded. Table 3.2 summarises 20 second runs of the corrected algorithm with the parameters of the original edition.

The table shows three things. First, almost half of the children (45–47 % on every benchmark) are copies of a sequence that is already in the population, mostly mutations that changed nothing and crossovers of equal parents; their evaluation brings no new information. Second, in at least half of the iterations the best sequence is already present in the initial random population (the median generation of the last improvement is 0 on seven of the eight benchmarks), so most of the 1000 generations of an iteration cannot change the step that is finally applied. Third, most iterations end with a step of equal fitness, a move on a plateau of the objective (96–100 % of the iterations on seven benchmarks, 67 %

Benchmark	iterations	children [%] duplicate	unchanged	as parent	iterations [%] impr. / equal / worse	best found in generation	evaluations per impr.
apte	2071	46.6	3.2	66.7	0.2 / 99.8 / 0.0	1 / 0	959 k
xerox	2312	45.4	2.7	59.5	0.4 / 98.1 / 1.5	38 / 0	476 k
hp	2194	44.7	3.4	62.2	0.0 / 100.0 / 0.0	1 / 0	–
ami33	2133	45.3	2.0	60.6	0.5 / 98.6 / 0.9	30 / 0	396 k
ami49	2061	45.5	1.9	59.3	1.6 / 67.3 / 31.0	195 / 32	113 k
n100	1674	45.1	1.6	63.7	2.0 / 97.4 / 0.5	31 / 0	91 k
n200	1370	46.4	1.0	59.7	3.4 / 96.3 / 0.3	30 / 0	54 k
n300	1131	46.7	0.9	59.5	2.7 / 95.9 / 1.3	42 / 0	68 k

Table 3.2: Behaviour of the genetic algorithm of POEMS ($G = 1000$, $N = 3$, $S = 50$, 20 s runs, seed 1). Children: duplicates of an individual of the population (same enabled actions), children that do not change the prototype, and children with the same fitness as a parent. Iterations: the best sequence of the iteration improves the prototype, has the same fitness, or is worse. Best found in generation: mean / median generation of the last improvement of the best individual of an iteration.

on ami49); only 0–3.4 % of them improve the prototype, which costs 54 000 to 4 million evaluations per improvement. Meanwhile a single random action improves the prototype much more often (Section 5.4). These observations motivate both the improvements of POEMS below and the new algorithm.

3.5 Improvements of POEMS

The following modifications are optional switches of the implementation (Appendix A); Section 5.3 evaluates each of them separately, together with smaller numbers of generations per iteration.

Array evaluation. Each action can also be applied in place to an array representation of the B*-Tree (Section 4.4). The prototype is converted once per iteration and every sequence is evaluated on a copy of its arrays; the identity check compares the trees exactly. The results are identical to the original evaluation, only faster.

Duplicate elimination. Each sequence has a 64-bit signature computed from its enabled actions (with the parameters of the symmetric exchange actions ordered). A child whose sequence is already in the population is discarded without being evaluated.

Early termination. An iteration ends after P generations without an improvement of the best individual (*patience*), or as soon as a sequence better than the prototype is found (*first improvement*).

```
1: β ← the best-fit prototype; β* ← β; history[0..L − 1] ← A(β)
2: for t = 0, 1, ... while time remains do
3:    γ ← a copy of β; apply k random actions to γ, k uniform in 1..N
4:    θ ← max(A(β), history[t mod L]) {late acceptance}
5:       or θ ← A(β) − T(t) ln u, u ∼ U(0, 1] {simulated annealing}
6:    if A(γ) ≤ θ (the placement stops as soon as the area exceeds θ) then
7:       β ← γ; if A(β) < A(β*) then β* ← β
8:    end if
9:    history[t mod L] ← A(β)
10: end for
11: return β*
```

Figure 3.3: Late acceptance POEMS (with the two acceptance criteria)

Late acceptance of prototypes. The result of an iteration is accepted if it is not worse than the prototype or than the prototype L iterations ago [2], which allows the search to leave local optima.

Move node action. The seventh action of Table 3.1.

3.6 Late Acceptance POEMS

The analysis of Section 3.4 suggests keeping what POEMS does well, improving a single prototype by short sequences of actions, and changing the search for the steps. Late acceptance POEMS (LA-POEMS, Fig. 3.3) applies a random action sequence to the prototype in every step and decides immediately whether the modified prototype replaces it. The niches of POEMS correspond to the lengths of the steps: a step consists of k enabled actions, with k chosen uniformly from $1, \ldots, N$. The types and the parameters of the actions are chosen uniformly at random.

3.6.1 Acceptance

Two acceptance criteria are compared. *Late acceptance hill climbing* [2] accepts the candidate if its area is not greater than the area of the current prototype or of the prototype L steps ago. Because the best history length depends on the number of steps that can be made in the given time, the implementation can also choose it automatically. It measures the evaluation speed during the first 2 % of the time and sets L to the expected number of steps divided by a constant C. *Simulated annealing* [14] accepts a deterioration

Δ with the probability $e^{-\Delta/T}$. The temperature decreases exponentially from T_0 to T_1 over the time budget, $T(t) = T_0(T_1/T_0)^{t/t_{\max}}$, with both temperatures given relative to the area of the modules. Both criteria are written as a threshold on the area of the candidate, which is known before the candidate is placed. This allows the early termination of the placement (Section 3.1.2).

3.6.2 Further Options

Four further options were evaluated (Section 5.4). With *weights*, the action types are chosen with given relative probabilities instead of uniformly. With *restarts*, the search restarts from the initial prototype (keeping the best solution) when the best solution of the current run has not improved for $R \cdot n$ steps. With *local partners*, the second module of a two-module action is, with a given probability, chosen by a random walk of one to three steps from the first module in the tree, which makes the step more local. With *adaptive action selection*, the action types are chosen in proportion to their success rates (the fraction of steps using the type that improved the prototype) in the last window of 10^5 steps, with a minimal probability of 5 % for each type.

The configuration chosen by the experiments of Section 5.4, which is the default of the program, uses simulated annealing with $T_0 = 10^{-2}$ and $T_1 = 10^{-5}$, steps of at most $N = 2$ actions and local partners with the probability 0.5.

3.6.3 Relation to POEMS

LA-POEMS keeps the representation, the actions, the prototype and the idea of improving it by steps of several actions, but replaces the evolution of the steps by sampling and a simple acceptance criterion. One step costs one evaluation instead of one iteration of about 1850 evaluations, and the prototype changes as soon as an acceptable step is found. The price is that steps are not refined: a step that is almost good is not recombined with other steps. Chapter 5 examines whether this price is worth paying.

Chapter 4

Implementation

The algorithms are implemented in Java. The original edition targeted Java 1.6 and the Eclipse IDE; the revised implementation builds with Maven on Java 8 or newer, which also runs the unit tests. This chapter describes the structure of the program, its performance engineering, the array-based B*-Tree, the defects found in the original implementation and the measures taken to make the results correct and reproducible.

4.1 Architecture

Fig. 4.1 shows the packages and the most important classes. The package `module` contains the geometric primitives (a rectangle, a module with a name and an orientation, a placed module). The package `btree` contains the immutable B*-Tree used by POEMS, the mutable array-based B*-Tree used by the fast evaluation and by LA-POEMS, and utilities for numbering the nodes, finding them, constructing the prototypes, placing the modules (`Placer` with the contour `PlacerCache`) and evaluating the placements. The package `poems` contains the actions, the action sequences, the genetic algorithm (`Population`, `Individual`) and the POEMS iterations; `poems.hybrid` contains LA-POEMS. The package `program` contains the command line programs, the import of benchmarks, the export of results (SVG, DOT and GNUPlot files), the settings and the logging.

4.2 The Original Design and its Cost

The original implementation follows a functional style: trees, modules and action sequences are immutable, and every action builds a new tree, copying the unchanged parts. This

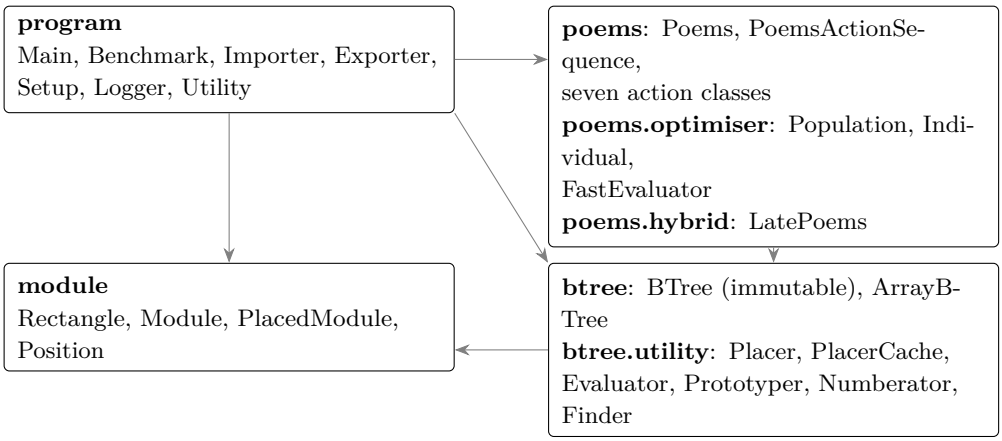


Figure 4.1: Packages and the most important classes (arrows show dependencies)

Hotspot	Share of time
contour as a `TreeSet` of all placed modules (`PlacerCache`)	40 %
tree iteration building a `Stack` and a `LinkedList` (`BTree.iterator`)	17 %
formatting of discarded debug messages	11 %

Table 4.1: Profile of the original implementation (n100, sampling profiler)

makes the actions easy to reason about and to test, and the code is well structured and documented. Its performance, however, matters a lot: the algorithm evaluates millions of candidate floorplans, and the quality of the result within a given time grows with their number.

Profiling a run on n100 showed that most of the time was spent outside the algorithm itself (Table 4.1). The contour was implemented as a sorted set of all placed modules walked from the top. Every search for a node by its number, every containment test and every evaluation built a stack and a linked list of the whole tree. And debug messages were formatted into strings, including complete action sequences, even though verbose mode was off and the strings were discarded.

4.3 Speed-ups that Do Not Change the Results

The following changes keep every computed value, including the sequence of random numbers, unchanged:

- the contour is a linked list of segments stored in arrays, and the placement passes the segment of the parent to its children, which makes the placement amortised linear (Section 3.1);

- the fitness is computed from the bounding box only; the tree of placed modules is constructed only when it is needed (for the export of the final result);
- finding a node, testing containment and evaluating a placement use recursion without allocating memory, while the iteration order of the trees is kept;
- debug messages are formatted only in verbose mode.

For the same seed and the same number of iterations, the optimised program produces exactly the same results as the original one, which was checked on six benchmarks with two seeds each. Section 5.2 reports the speed-ups.

4.4 The Array B*-Tree

The immutable trees still allocate a new tree for every action. The class `ArrayBTree` stores a B*-Tree in integer arrays: the left child, the right child, the parent and the module of every node, the node of every module, and the orientation of every module. The actions modify the arrays in place:

- flip, mirror and rotate visit the affected subtree;
- exchange value swaps two modules in $O(1)$;
- exchange node and hang node check the ancestor relation by walking up to the root, and relink the nodes in $O(1)$;
- move node removes and inserts a node in $O(1)$, plus the walk to the top of the rightmost branch of the left child.

The placement is an iterative depth-first traversal with an explicit stack and the array contour, and it supports the early termination of Section 3.1.2. A copy of the tree is a copy of six arrays. Two trees are compared by a simultaneous depth-first traversal comparing the modules, their orientations and the presence of children, so the comparison does not depend on how the nodes are numbered internally.

The array actions must behave exactly like the original immutable actions. A unit test applies 10 000 random actions of all seven types, with random parameters, to both representations of random instances. After every action, it checks that the trees are equal and that the placed areas agree. A second test applies 10 000 random action sequences to random prototypes and checks that the fitness computed on the array representation, including the identity check, equals the fitness computed on the immutable trees. Complete runs of POEMS with both evaluations give identical results for the same seed and number of iterations.

Defect	Consequence and fix
identity check ignored orientations	a sequence that only rotated modules was considered a no-op and got the minimal fitness, so a module was never rotated alone; the comparison now includes the orientation
best-fit prototype	iterated a priority queue, which does not iterate in priority order, so it did not choose the widest fitting module; it now uses a sorted list
first iteration	its result was accepted even if worse than the prototype; the prototype is now evaluated and is the initial best solution
increment of node numbers	wrapped around modulo $n - 1$ instead of n, so mutations never reached the last node
infinite loop in `shuffleArray`	the order mutation of a sequence of length one searched forever for two different positions, so POEMS hung with $N = 1$; arrays shorter than two are not shuffled
`NumberedValue.hashCode`	was not consistent with `equals`
average fitness statistic	was a running pairwise average instead of the mean

Table 4.2: Defects of the original implementation

4.5 Defects Found and Fixed

Table 4.2 lists the defects found in the original implementation. The first four changed the behaviour of the algorithm described in the original edition, the fifth made the program hang for $N = 1$, and the last two did not influence the search. The identity check had the largest effect: it made pure rotations impossible, although rotation is one of the most useful moves (Section 5.4). In addition, twelve unit tests failed on Java 9 and newer because of a type cast in a test utility, and one test expected a prototype that the code never produced; the test suite now passes completely (80 tests).

4.6 Correctness and Reproducibility

Besides the unit tests, every result reported in Chapter 5 was checked by an independent validator. It checks that every module of the instance appears exactly once, with its original

or rotated dimensions, at non-negative coordinates, and that no two modules overlap. All runs passed. The random number generator can be seeded, so a run with a given seed and a given number of iterations or evaluations is reproducible; runs with a time limit are reproducible up to the number of steps made within the limit. A benchmark runner (`program.Benchmark`) solves one instance with a given seed, time limit and parameters, validates the result and prints a line of comma-separated values. It can also write the time and quality of every improvement of the best solution (the convergence trace) and the coordinates of the modules of the final floorplan. The scripts used for the experiments are described in Appendix A.

Chapter 5

Experimental Evaluation

This chapter answers the research questions of Chapter 1. Section 5.1 describes the methodology common to all experiments. Section 5.2 measures the effect of the implementation (RQ1). Section 5.3 evaluates the corrections and improvements of POEMS (RQ2), and Section 5.4 the design of LA-POEMS (RQ3). Section 5.5 compares the algorithms with each other, with the original edition and with the literature (RQ4). The chapter closes with the threats to validity.

5.1 Methodology

Budget. Every run has a fixed wall-clock time limit, measured from the start of the optimisation, after the benchmark has been loaded; it includes the just-in-time compilation of the program, which takes place during the run. Equal time, rather than an equal number of evaluations, is the fair basis for comparing implementations and algorithms whose steps have different costs. The results of the original edition were reported after a fixed number of iterations, so their running times are given with them.

Seeds. Every run has its own seed of the random number generator. The screening experiments (Sections 5.2–5.4) use the seeds 1–8 or 1–6. The final comparison (Section 5.5) uses the fresh seeds 101–108 on the tuning benchmarks and 101–105 on the held-out benchmarks; these seeds were not used when the configurations were chosen.

Benchmarks. The eight benchmarks of Table 2.2 were used for development and tuning; apte is omitted from most screening tables because every variant finds the same floorplan (0.78 %). The six held-out GSRC instances were used only in the final comparison.

Measures. The quality of a run is the dead space of its best floorplan (Chapter 2). The tables report the mean and the population standard deviation over the seeds, and the best run in parentheses. Every floorplan was validated independently (Chapter 4).

Statistics. Two variants are compared by the two-sided Mann-Whitney U test [20] at the level 0.05 on the per-seed results. In the tables, a **bold** entry is significantly better and an entry marked ▲ significantly worse than the first (reference) column of the table, unless the caption says otherwise. The final comparison also reports the Vargha-Delaney effect size $\hat{A}_{12}$ [24], the probability that a run of one algorithm ends with less dead space than a run of the other (ties count one half). To compare configurations across benchmarks, the *score* of a configuration is the geometric mean over the benchmarks of its mean dead space divided by the best mean dead space among the compared configurations; the best possible score is 1.

Platform. All experiments ran on a virtual machine with four cores of an Intel Xeon at 2.8 GHz and OpenJDK 21, with the serial garbage collector and a heap of at most 1 GB. Three runs ran in parallel, so that every run had a core of its own and the fourth core was left for the system; runs that are compared with each other always ran under the same conditions. The machine is about 1.7 times faster than the 2 GHz machine of the original edition: the original program needed 131.7 s for 1000 iterations on n100 there (Table 5.3 of the original edition) and needs 77 s here (24 000 evaluations per second, mean of 8 runs).

5.2 RQ1: The Cost of the Implementation

Placement. Fig. 5.1 shows the time of one placement of a complete B*-Tree with randomly sized modules. The original contour, a sorted set of all placed modules, is superlinear: 256 times more modules take 3200 times longer, roughly $n^{1.46}$. The original edition measured the same behaviour (Table 5.1 of the original edition shows growth by a factor of 1.9–3.1 per doubling) but concluded that it was close to linear. The array contour is linear, as the analysis of Section 3.1 predicts. Together with the array B*-Tree it is 7 times faster than the original placement for 100 modules and 99 times faster for 25 600 modules.

Evaluation speed. Table 5.1 shows how many floorplans per second the algorithms evaluate on the benchmarks. The optimisations of the immutable implementation, which do not change the results, make POEMS 4–8 times faster; the speed-up grows with the number of modules because the original placement is superlinear. The corrections do not change the speed. The array evaluation adds another factor of 1.7 to 3.7, so that POEMS evaluates

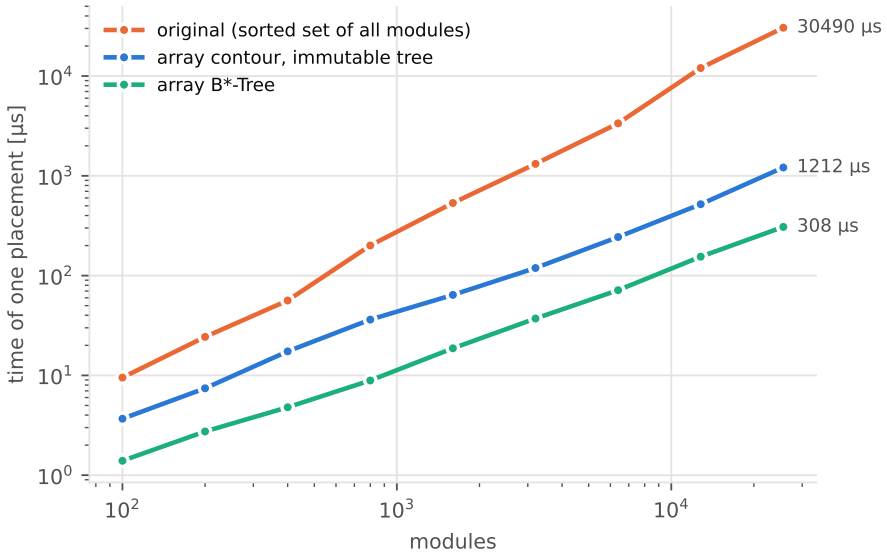


Figure 5.1: Time of one placement of a complete B*-Tree with random module sizes: the original contour, the array contour with the immutable tree, and the array B*-Tree (median of 9 batches after a warm-up; logarithmic axes)

7.5 to 29 times more floorplans per second than the original implementation. LA-POEMS, which does not create any objects per step and stops the placement of rejected candidates early, makes 9 to 37 times more evaluations per second than the original implementation, but many of them are only partial placements.

Quality. Table 5.2 compares the original implementation with the optimised one. Both run the same algorithm with the same seeds, so the only difference is the number of evaluations made in the 30 seconds. The faster implementation finds significantly better floorplans on all five benchmarks with more than 30 modules; for example, the mean dead space on n300 drops from 13.1 % to 9.0 %.

Answer to RQ1. Most of the running time of the original implementation was overhead that did not influence the computed results: the superlinear contour, the allocation of temporary lists for every tree traversal and the formatting of discarded messages. Removing it speeds up POEMS 4–8 times with identical results, which translates into significantly better floorplans within the same time on all larger benchmarks. The quality reported for an evolutionary algorithm therefore depends strongly on its implementation, and comparisons with other algorithms must be read with this in mind.

Benchmark	Original	Optimised	Corrected	Array	LA-POEMS
xerox	184	796 (4.3×)	791 (4.3×)	1379 (7.5×)	1642 (8.9×)
hp	165	791 (4.8×)	745 (4.5×)	1319 (8.0×)	1680 (10.2×)
ami33	54	383 (7.1×)	382 (7.0×)	848 (15.7×)	1117 (20.6×)
ami49	54	297 (5.5×)	263 (4.9×)	698 (13.0×)	869 (16.2×)
n100	24	156 (6.5×)	147 (6.1×)	447 (18.6×)	546 (22.7×)
n200	10	79 (7.8×)	73 (7.3×)	253 (25.1×)	323 (32.1×)
n300	6	48 (7.8×)	49 (8.0×)	179 (29.1×)	227 (36.8×)

Table 5.1: Evaluations per second (thousands) of the original implementation of POEMS, the optimised implementation, the corrected one, the corrected one with the array evaluation, and LA-POEMS, whose evaluations stop early when the candidate is rejected (mean of the 30 s runs; speed-up relative to the original implementation in parentheses)

Benchmark	Original	Optimised
apte	$0.78_{\pm 0.00}$ (0.78)	$0.78_{\pm 0.00}$ (0.78)
xerox	$3.41_{\pm 0.58}$ (2.50)	$3.07_{\pm 0.41}$ (2.34)
hp	$5.23_{\pm 1.05}$ (3.88)	$4.43_{\pm 1.08}$ (2.28)
ami33	$4.19_{\pm 0.73}$ (2.91)	$\mathbf{2.58_{\pm 0.46}}$ (**1.73**)
ami49	$4.90_{\pm 0.71}$ (3.70)	$\mathbf{3.37_{\pm 0.64}}$ (**2.31**)
n100	$6.88_{\pm 1.42}$ (5.61)	$\mathbf{4.15_{\pm 0.60}}$ (**3.49**)
n200	$11.97_{\pm 1.15}$ (10.30)	$\mathbf{7.30_{\pm 0.47}}$ (**6.67**)
n300	$13.09_{\pm 1.03}$ (12.02)	$\mathbf{9.00_{\pm 0.51}}$ (**7.99**)

Table 5.2: Dead space [%] of the original implementation and of the implementation optimised without changing the results (same algorithm and seeds, 30 s, 8 seeds)

Benchmark	Before corrections	After corrections
apte	$0.78_{\pm 0.00}$ (0.78)	$0.78_{\pm 0.00}$ (0.78)
xerox	$3.07_{\pm 0.41}$ (2.34)	$2.79_{\pm 0.37}$ (2.34)
hp	$4.43_{\pm 1.08}$ (2.28)	$3.79_{\pm 1.76}$ (1.32)
ami33	$2.58_{\pm 0.46}$ (1.73)	$2.98_{\pm 0.59}$ (2.22)
ami49	$3.37_{\pm 0.64}$ (2.31)	$2.95_{\pm 0.39}$ (2.34)
n100	$4.15_{\pm 0.60}$ (3.49)	$5.00_{\pm 0.60}$ (4.43)▲
n200	$7.30_{\pm 0.47}$ (6.67)	$7.66_{\pm 0.42}$ (6.73)
n300	$9.00_{\pm 0.51}$ (7.99)	$\mathbf{8.12_{\pm 0.56}}$ (**7.14**)

Table 5.3: Dead space [%] before and after the corrections of Section 4.5 (30 s, 8 seeds)

5.3 RQ2: Corrections and Improvements of POEMS

5.3.1 Effect of the Corrections

Table 5.3 compares POEMS before and after the corrections of Section 4.5, both with the fast immutable implementation. The corrections change the behaviour of the algorithm in several ways at once, and their combined effect within 30 seconds is mixed. The dead space is significantly smaller on n300 and significantly larger on n100; on the remaining benchmarks the differences are not significant (smaller means on xerox, hp and ami49, larger on ami33 and n200).

Two of the corrections change the search itself. The corrected prototype is a different, deterministic starting point of all runs. The corrected identity check makes steps that only rotate modules possible. To separate them on n100, each was reverted alone. With the original prototype, the mean dead space on n100 is 4.49 % (8 seeds); with the original identity check, it is 4.36 %. Both lie between the results before (4.15 %) and after (5.00 %) the corrections, so both contribute to the difference, and neither difference is significant on its own.

Table 5.4 shows how the identity check changes the iterations. With the original check, a step that only rotates modules is rejected, so the genetic algorithm has to find a structural change. When it does not find a good one, the iteration ends with a worse step, which is not applied (9–16 % of the iterations on ami33, n100 and n300). With the corrected check, a rotation of a module that does not touch the boundary of the floorplan is a step of equal fitness, which is found almost immediately (on ami33, n100 and n300, the mean generation of the best individual drops from 128–175 to 23–32) and accepted. The share of improving iterations does not change. In other words, the correction turns rejected iterations into

Benchmark	iterations improving / equal / worse [%] rotation ignored	corrected	generation of the best (mean) rotation ignored	corrected
xerox	0.4 / 97.8 / 1.8	0.4 / 98.0 / 1.5	50	40
ami33	0.9 / 87.5 / 11.6	0.6 / 99.0 / 0.4	138	23
n100	3.0 / 88.0 / 9.0	2.9 / 96.9 / 0.2	128	24
n300	5.3 / 78.9 / 15.9	4.9 / 94.9 / 0.2	175	32

Table 5.4: Outcome of the iterations of POEMS with the original identity check, which ignores the orientation of the modules, and with the corrected one (20 s, seed 1)

moves along plateaus. Whether such a move helps depends on the instance, which agrees with the mixed results of Table 5.3.

The corrections are kept in all further experiments: they make the implementation agree with the algorithm described in Chapter 3, and the remaining defects (the hang for $N = 1$, the unreachable last node, the first iteration that could make the prototype worse) are plain errors.

5.3.2 Improvements of the Genetic Algorithm

All variants in this subsection use the corrections and the array evaluation, which gives the same results as the immutable evaluation and makes POEMS another 1.7 to 3.7 times faster (Table 5.1). Each variant changes one setting of the reference configuration, which uses the parameters of the original edition ($G = 1000$, $N = 3$, $S = 50$). Table 5.5 shows the results. The rule for the *improved POEMS* of the final comparison was fixed before the experiment: it combines all modifications whose score is better than that of the reference (of the reduced numbers of generations, only the better one).

The number of generations per iteration matters most. With $G = 100$ instead of 1000, an iteration costs 320 instead of 1856 evaluations, POEMS makes 5.6 times more iterations in the same time, and its dead space is significantly smaller on ami33, n100, n200 and n300. This agrees with the analysis of Section 3.4: the best step of an iteration is usually found early, so the later generations mostly delay the next iteration. $G = 300$ lies in between. Discarding duplicate children saves 42 % of the evaluations of an iteration and gives up to 45 % more iterations on the large instances, where it is slightly better, but it is worse on xerox, hp and ami49; none of the differences is significant. A plausible explanation is that duplicates of good sequences are not only waste: they increase the share of the good sequences in the population, so discarding them weakens the selection. The move node action does not help either. Ending an iteration after 100 generations without improvement (patience) has an effect similar to $G = 100$ (about 330 evaluations per iteration) and is significantly better on ami33, n200 and n300. Ending it at the first improvement of the prototype is worse: because few iterations improve the prototype, the iterations still run

Benchmark	reference	$G = 300$	$G = 100$	no dupl.	move	patience	first	LA 20
xerox	$2.66_{\pm 0.35}$	$2.61_{\pm 0.38}$	$2.74_{\pm 0.34}$	$2.92_{\pm 0.37}$	$3.01_{\pm 0.32}$	$2.93_{\pm 0.43}$	$2.94_{\pm 0.38}$	$2.70_{\pm 0.34}$
hp	$2.41_{\pm 1.59}$	$4.23_{\pm 0.78}$	$2.14_{\pm 1.17}$	$3.23_{\pm 1.69}$	$2.64_{\pm 1.87}$	$2.83_{\pm 1.59}$	$2.41_{\pm 1.59}$	$3.81_{\pm 1.35}$
ami33	$2.74_{\pm 0.47}$	$\mathbf{1.89_{\pm 0.48}}$	$\mathbf{1.92_{\pm 0.18}}$	$2.58_{\pm 0.29}$	$2.27_{\pm 0.45}$	$\mathbf{1.83_{\pm 0.37}}$	$2.50_{\pm 0.16}$	$2.74_{\pm 0.47}$
ami49	$2.33_{\pm 0.23}$	$2.40_{\pm 0.28}$	$2.45_{\pm 0.35}$	$2.90_{\pm 0.48}$	$2.82_{\pm 0.65}$	$2.47_{\pm 0.23}$	$2.71^{\blacktriangle}_{\pm 0.29}$	$2.55_{\pm 0.28}$
n100	$3.72_{\pm 0.45}$	$\mathbf{3.09_{\pm 0.28}}$	$\mathbf{2.97_{\pm 0.25}}$	$3.53_{\pm 0.62}$	$3.79_{\pm 0.79}$	$3.16_{\pm 0.25}$	$3.73_{\pm 0.39}$	$3.83_{\pm 0.58}$
n200	$5.60_{\pm 0.51}$	$\mathbf{4.99_{\pm 0.24}}$	$\mathbf{4.47_{\pm 0.11}}$	$5.57_{\pm 0.49}$	$6.20_{\pm 0.56}$	$\mathbf{4.33_{\pm 0.42}}$	$5.89_{\pm 0.34}$	$6.18_{\pm 0.42}$
n300	$6.51_{\pm 0.53}$	$5.86_{\pm 0.58}$	$\mathbf{5.57_{\pm 0.40}}$	$6.19_{\pm 0.57}$	$6.29_{\pm 0.40}$	$\mathbf{5.42_{\pm 0.43}}$	$6.67_{\pm 0.32}$	$6.65_{\pm 0.29}$
score	1.188	1.154	1.030	1.265	1.240	1.077	1.229	1.315

Table 5.5: Modifications of the genetic algorithm of POEMS, each applied alone to the reference configuration (array evaluation, $G = 1000$, $N = 3$, $S = 50$): fewer generations per iteration, discarding duplicate children, the move node action, ending an iteration after 100 generations without improvement or at the first improvement of the prototype, and late acceptance of prototypes with a history of 20 iterations (dead space [%], 30 s, seeds 1–6)

almost all 1000 generations (about 1840 instead of 1856 evaluations per iteration), and when one improves it, the improving step is applied before a better one can be found. Late acceptance of prototypes is worse as well. It rarely has an effect, because almost all iterations end with a step of equal fitness, so the history mostly contains the current fitness; on ami33 it did not change the result of any run.

By the rule stated above, the improved POEMS combines $G = 100$ with the patience of 100 generations. With $G = 100$, however, the patience never ends an iteration early, so the improved POEMS is simply POEMS with the corrections, the array evaluation and $G = 100$, and its screening results are those of the column $G = 100$ of Table 5.5.

Answer to RQ2. The genetic algorithm of POEMS uses its evaluations inefficiently. Almost half of its children duplicate individuals of the population, the best step of an iteration is usually present in the initial random population, and only a few percent of the iterations improve the prototype. The corrections of the defects change the behaviour of the algorithm, mainly by allowing pure rotations, but their combined effect on the results is mixed. Of the evaluated modifications, only those that shorten the iterations help: with $G = 100$, POEMS is significantly better on four of the seven benchmarks, and with the patience of 100 generations on three. Duplicate elimination, the move node action, the first-improvement rule and late acceptance of prototypes do not improve it.

5.4 RQ3: Design of LA-POEMS

The design of LA-POEMS was explored in a sequence of screening experiments (30 s, seeds 1–6, the seven benchmarks except apte), starting from late acceptance with the history

	improving steps per 10^4			steps of equal area [%]		
Action	ami33	n100	n300	ami33	n100	n300
flip	1.03	5.71	7.02	5.6	3.3	9.7
rotate	0.63	3.78	3.37	2.2	1.6	5.2
mirror	0.02	0.31	0.28	27.8	25.6	19.6
exchange value	0.48	4.79	3.93	3.2	1.3	3.1
exchange node	0.15	1.31	0.28	37.0	22.8	12.6
hang node	0.06	0.62	0.57	28.4	17.2	9.7
move node	0.08	1.54	1.98	2.4	0.8	0.7

Table 5.6: Outcome of the steps of LA-POEMS that consist of a single action, by action type (late acceptance with $L = 1000$, steps of at most three actions of uniformly chosen types, 10 s, seed 1; about 10^5–$5 \cdot 10^5$ single-action steps per type)

length $L = 1000$ and steps of at most three actions.

5.4.1 Single Actions

Table 5.6 shows how often a step consisting of a single action of each type improves the prototype or leaves its area unchanged. Rotating a module (flip), exchanging two modules and transposing a subtree (rotate) improve the prototype most often; on n100 and n300, moving a single node is next. Mirroring, exchanging subtrees and hanging subtrees rarely improve the prototype, but 10–37 % of them do not change the area. They are moves along plateaus: they change the structure of the tree without affecting the enclosing rectangle, which keeps the search moving when no improving step is available.

5.4.2 Acceptance

Table 5.7 compares the acceptance criteria. With late acceptance, the history length sets how quickly the search converges. A long history ($L = 5000$) is significantly worse on the three largest instances, on which the search does not converge within 30 seconds; a short one ($L = 200$) is significantly better on n200 and n300 and slightly worse on xerox, ami33 and ami49. The automatic history length adapts to the speed of the evaluation and to the time budget. It finds the best known floorplan of hp (1.32 %, the area reported for the B*-Tree in [4]) in every run and is good on the large instances, but it gets stuck on xerox. Simulated annealing with a high initial temperature ($T_0 = 10^{-2}$ of the area of the modules) has the best score. It finds the best known floorplan of xerox (2.30 %) in five of six runs and that of hp in four, and it is significantly better than late acceptance with

	late acceptance L			automatic C		annealing T_0		
Benchmark	1000	200	5000	$2{\cdot}10^4$	$4{\cdot}10^4$	10^{-3}	$3{\cdot}10^{-3}$	10^{-2}
xerox	$2.97_{\pm 0.32}$	$3.13_{\pm 0.68}$	$2.93_{\pm 0.43}$	$3.18_{\pm 0.64}$	$3.18_{\pm 0.64}$	$2.90_{\pm 0.36}$	$\mathbf{2.32}_{\pm \mathbf{0.02}}$	$\mathbf{2.31}_{\pm \mathbf{0.02}}$
hp	$3.23_{\pm 1.45}$	$2.66_{\pm 1.34}$	$3.09_{\pm 1.27}$	$\mathbf{1.32}_{\pm \mathbf{0.00}}$	$\mathbf{1.32}_{\pm \mathbf{0.00}}$	$3.10_{\pm 1.30}$	$2.97_{\pm 1.17}$	$1.79_{\pm 0.72}$
ami33	$1.43_{\pm 0.34}$	$1.57_{\pm 0.40}$	$1.54_{\pm 0.27}$	$1.46_{\pm 0.30}$	$1.59_{\pm 0.25}$	$1.54_{\pm 0.27}$	$1.79_{\pm 0.48}$	$1.74_{\pm 0.41}$
ami49	$1.86_{\pm 0.35}$	$1.96_{\pm 0.15}$	$1.79_{\pm 0.23}$	$1.88_{\pm 0.28}$	$1.96_{\pm 0.18}$	$1.77_{\pm 0.28}$	$2.13_{\pm 0.26}$	$2.03_{\pm 0.25}$
n100	$2.47_{\pm 0.16}$	$2.48_{\pm 0.10}$	$2.72^{\blacktriangle}_{\pm 0.12}$	$2.37_{\pm 0.11}$	$2.44_{\pm 0.23}$	$2.30_{\pm 0.21}$	$2.34_{\pm 0.27}$	$2.36_{\pm 0.13}$
n200	$4.83_{\pm 0.41}$	$\mathbf{4.16}_{\pm \mathbf{0.26}}$	$7.36^{\blacktriangle}_{\pm 0.25}$	$4.38_{\pm 0.23}$	$\mathbf{4.03}_{\pm \mathbf{0.22}}$	$\mathbf{4.11}_{\pm \mathbf{0.35}}$	$\mathbf{4.11}_{\pm \mathbf{0.34}}$	$\mathbf{3.87}_{\pm \mathbf{0.29}}$
n300	$6.46_{\pm 0.33}$	$\mathbf{5.17}_{\pm \mathbf{0.20}}$	$9.54^{\blacktriangle}_{\pm 0.46}$	$\mathbf{5.55}_{\pm \mathbf{0.31}}$	$\mathbf{5.22}_{\pm \mathbf{0.09}}$	$\mathbf{5.45}_{\pm \mathbf{0.18}}$	$\mathbf{5.31}_{\pm \mathbf{0.35}}$	$\mathbf{4.59}_{\pm \mathbf{0.26}}$
score	1.299	1.232	1.473	1.112	1.113	1.219	1.229	1.099

Table 5.7: Acceptance criteria of LA-POEMS: late acceptance with a fixed history length L or with the automatic length (ratio C), and simulated annealing with the initial temperature T_0 relative to the area of the modules (final temperature 10^{-6}, or 10^{-5} for $T_0 = 10^{-2}$); steps of at most three actions (dead space [%], 30 s, seeds 1–6)

$L = 1000$ on xerox, n200 and n300; on ami33 and ami49 it is slightly, but not significantly, worse. Lower initial temperatures are better on ami49 and n100, but they get stuck on hp as often as late acceptance does.

5.4.3 Steps and Other Options

Table 5.8 varies the steps of LA-POEMS with late acceptance. Steps of a single action are significantly better on n100 and slightly better on n200 and n300, but much worse on xerox, hp and ami33, which need the compound steps of POEMS to escape from local optima. Steps of at most two actions are significantly better on n100 and similar elsewhere. Weights favouring the frequently improving action types help the large instances and hurt ami33 and ami49. Restarts solve the small instances: hp reaches 1.32 % in every run and xerox 2.34–2.37 %; on the large instances, they do not help. Choosing the second module of an action close to the first one has no consistent effect.

Table 5.9 varies the options of LA-POEMS with simulated annealing. As with late acceptance, steps of a single action are significantly better on n100 and n200 and significantly worse on hp. Local partners are significantly better on n200, and together with steps of at most two actions they are significantly better on n200 and n300. The adaptive selection of the action types is significantly worse on n300, presumably because it suppresses the action types that rarely improve the prototype but move it along plateaus (Table 5.6). For comparison, the table also shows late acceptance with the automatic history length and restarts, the best configuration with late acceptance: it solves hp in every run, but it is significantly worse on xerox and n300.

Benchmark	base	$k = 1$	$k \leq 2$	weights	restarts	local 0.5	local 0.25
xerox	$2.97_{\pm 0.32}$	$4.70^{\blacktriangle}_{\pm 1.46}$	$2.93_{\pm 0.45}$	$2.87_{\pm 0.37}$	$2.35_{\pm 0.01}$	$3.09_{\pm 0.72}$	$3.04_{\pm 0.31}$
hp	$3.23_{\pm 1.45}$	$5.14_{\pm 1.24}$	$3.84_{\pm 1.33}$	$2.57_{\pm 1.25}$	$\mathbf{1.32_{\pm 0.00}}$	$2.33_{\pm 1.08}$	$3.13_{\pm 1.32}$
ami33	$1.43_{\pm 0.34}$	$2.06_{\pm 0.78}$	$1.43_{\pm 0.28}$	$1.70_{\pm 0.56}$	$1.61_{\pm 0.29}$	$1.39_{\pm 0.32}$	$1.60_{\pm 0.42}$
ami49	$1.86_{\pm 0.35}$	$1.92_{\pm 0.27}$	$1.74_{\pm 0.17}$	$2.06_{\pm 0.49}$	$1.86_{\pm 0.35}$	$2.20_{\pm 0.51}$	$2.14_{\pm 0.32}$
n100	$2.47_{\pm 0.16}$	$\mathbf{1.91_{\pm 0.27}}$	$\mathbf{2.18_{\pm 0.12}}$	$\mathbf{2.22_{\pm 0.21}}$	$2.63_{\pm 0.23}$	$2.53_{\pm 0.32}$	$2.37_{\pm 0.13}$
n200	$4.83_{\pm 0.41}$	$4.48_{\pm 0.29}$	$4.37_{\pm 0.37}$	$4.29_{\pm 0.20}$	$4.83_{\pm 0.41}$	$4.76_{\pm 0.22}$	$4.49_{\pm 0.25}$
n300	$6.46_{\pm 0.33}$	$6.18_{\pm 0.39}$	$6.25_{\pm 0.23}$	$\mathbf{5.77_{\pm 0.28}}$	$6.35_{\pm 0.31}$	$6.31_{\pm 0.22}$	$6.27_{\pm 0.26}$
score	1.277	1.460	1.246	1.219	1.110	1.247	1.294

Table 5.8: Options of LA-POEMS with late acceptance ($L = 1000$): steps of exactly one action and of at most two actions instead of three, weights of the action types favouring flip, rotate, exchange value and move node (3:2:1:3:1:1:2), restarts after $5 \cdot 10^4 n$ steps without improvement, and local partners with the probability 0.5 and 0.25 (dead space [%], 30 s, seeds 1–6)

Benchmark	base	$k = 1$	$k \leq 2$	local	$k \leq 2$, local	adaptive	LA, restarts
xerox	$2.31_{\pm 0.02}$	$2.35_{\pm 0.06}$	$2.32_{\pm 0.02}$	$2.34_{\pm 0.06}$	$2.33_{\pm 0.02}$	$2.32_{\pm 0.02}$	$2.39^{\blacktriangle}_{\pm 0.06}$
hp	$1.79_{\pm 0.72}$	$3.66^{\blacktriangle}_{\pm 0.74}$	$2.20_{\pm 0.98}$	$1.72_{\pm 0.89}$	$2.38_{\pm 1.08}$	$1.72_{\pm 0.89}$	$1.32_{\pm 0.00}$
ami33	$1.74_{\pm 0.41}$	$1.61_{\pm 0.34}$	$1.60_{\pm 0.36}$	$1.65_{\pm 0.55}$	$1.39_{\pm 0.28}$	$1.68_{\pm 0.32}$	$1.65_{\pm 0.11}$
ami49	$2.03_{\pm 0.25}$	$1.72_{\pm 0.26}$	$1.77_{\pm 0.26}$	$2.35_{\pm 0.27}$	$2.23_{\pm 0.29}$	$2.25_{\pm 0.29}$	$1.95_{\pm 0.20}$
n100	$2.36_{\pm 0.13}$	$\mathbf{2.05_{\pm 0.18}}$	$2.42_{\pm 0.20}$	$2.39_{\pm 0.15}$	$2.41_{\pm 0.22}$	$2.63_{\pm 0.26}$	$2.53_{\pm 0.18}$
n200	$3.87_{\pm 0.29}$	$\mathbf{3.19_{\pm 0.41}}$	$3.47_{\pm 0.53}$	$\mathbf{3.37_{\pm 0.28}}$	$\mathbf{2.98_{\pm 0.50}}$	$3.58_{\pm 0.40}$	$4.09_{\pm 0.19}$
n300	$4.59_{\pm 0.26}$	$4.21_{\pm 0.27}$	$4.56_{\pm 0.39}$	$4.51_{\pm 0.21}$	$\mathbf{3.84_{\pm 0.31}}$	$5.04^{\blacktriangle}_{\pm 0.26}$	$5.31^{\blacktriangle}_{\pm 0.17}$
score	1.200	1.213	1.182	1.187	1.157	1.225	1.183

Table 5.9: Options of LA-POEMS with simulated annealing ($T_0 = 10^{-2}$, $T_1 = 10^{-5}$, steps of at most three actions): steps of exactly one and of at most two actions, local partners with the probability 0.5 (alone and with steps of at most two actions), adaptive selection of the action types, and, for comparison, late acceptance with the automatic history ($C = 4 \cdot 10^4$) and restarts (dead space [%], 30 s, seeds 1–6)

Benchmark	POEMS	first design	final	$\hat{A}_{12}$ vs. POEMS first design	final
xerox	$2.66_{\pm 0.35}$ (2.34)	$2.97_{\pm 0.32}$ (2.30)	$\mathbf{2.33_{\pm 0.02}}$ (**2.30**)	0.33	0.92
hp	$2.41_{\pm 1.59}$ (1.32)	$3.23_{\pm 1.45}$ (1.32)	$2.38_{\pm 1.08}$ (1.32)	0.38	0.50
ami33	$2.74_{\pm 0.47}$ (2.22)	$\mathbf{1.43_{\pm 0.34}}$ (**0.90**)	$\mathbf{1.39_{\pm 0.28}}$ (**1.09**)	1.00	1.00
ami49	$2.33_{\pm 0.23}$ (2.08)	$1.86_{\pm 0.35}$ (1.28)	$2.23_{\pm 0.29}$ (1.92)	0.83	0.68
n100	$3.72_{\pm 0.45}$ (2.99)	$\mathbf{2.47_{\pm 0.16}}$ (**2.33**)	$\mathbf{2.41_{\pm 0.22}}$ (**2.08**)	1.00	1.00
n200	$5.60_{\pm 0.51}$ (4.88)	$\mathbf{4.83_{\pm 0.41}}$ (**4.28**)	$\mathbf{2.98_{\pm 0.50}}$ (**2.18**)	0.86	1.00
n300	$6.51_{\pm 0.53}$ (5.77)	$6.46_{\pm 0.33}$ (5.99)	$\mathbf{3.84_{\pm 0.31}}$ (**3.39**)	0.46	1.00

Table 5.10: POEMS and LA-POEMS on the same array implementation: POEMS with the parameters of the original edition, the first design of LA-POEMS (late acceptance, $L = 1000$, steps of at most three actions) and the final configuration of LA-POEMS; both LA-POEMS columns are compared with POEMS (dead space [%], 30 s, seeds 1–6)

5.4.4 The Final Configuration

The score was fixed in advance as the criterion for choosing the configuration (Section 5.1). The configuration with the best score among the 20 screened ones is simulated annealing with $T_0 = 10^{-2}$ and $T_1 = 10^{-5}$, steps of at most $N = 2$ actions and local partners with the probability 0.5; it is the default configuration of the program. Its advantage over the other configurations with annealing and over late acceptance with restarts is small and comes from the two largest instances, while late acceptance with restarts is more reliable on hp. Because the configuration was selected on the same seeds on which it was screened, its screening results are optimistically biased; the final comparison (Section 5.5) therefore uses new seeds and held-out instances.

5.4.5 LA-POEMS and POEMS at Equal Speed

Table 5.10 compares POEMS and LA-POEMS on the same array implementation with the same time, which isolates the effect of the genetic algorithm. The first design of LA-POEMS, late acceptance with $L = 1000$ and steps of at most three actions, which was not tuned, is already significantly better than POEMS on ami33, n100 and n200 ($\hat{A}_{12} = 0.86$–1); it is worse on xerox and hp and equal on n300, but not significantly. The final configuration is significantly better than POEMS on five of the seven instances ($\hat{A}_{12} \geq 0.92$) and not significantly different on hp and ami49. Because the final configuration was selected on these seeds, its advantage is overestimated here; Section 5.5 repeats the comparison with new seeds against the improved POEMS.

Answer to RQ3. Evolving the improvement steps by a genetic algorithm is not necessary. With the same actions, the same implementation and the same time, applying random steps immediately and accepting them by a simple criterion finds significantly better floorplans on most instances and significantly worse ones on none. The acceptance criterion is the most important design choice: simulated annealing with a high initial temperature is the most robust, while late acceptance with a fixed history length is good only for budgets that match the history length. Steps of at most two actions and local partners help on the large instances; steps of a single action fail on the small ones, which need compound steps to escape from local optima; weights and the adaptive selection of the action types do not help.

5.5 RQ4: Final Comparison

The final comparison runs three algorithms for 60 seconds with seeds that were not used before: the original implementation of POEMS with the parameters of the original edition (*original POEMS*), POEMS with the corrections, the array evaluation and the modifications chosen in Section 5.3 (*improved POEMS*), and LA-POEMS in the configuration chosen in Section 5.4. The held-out instances were not used before this comparison.

5.5.1 Tuning Benchmarks

Table 5.11 shows the results on the eight benchmarks used for tuning. All three algorithms find the same floorplan of apte (0.78 %) in every run. On the other seven benchmarks, LA-POEMS is significantly better than the original POEMS, with $\hat{A}_{12} \geq 0.98$: almost every run of LA-POEMS ends with less dead space than every run of the original algorithm. It reduces the mean dead space by 28 % on xerox, by 57–65 % on hp, ami33, ami49 and n100 and by 69–73 % on n200 and n300, from 10–11 % to 2.8–3.4 %. It finds the best known floorplan of hp (1.32 %) in seven of eight runs and that of xerox (2.30 %) in four. The improved POEMS, which differs from the original algorithm by the corrections, the faster evaluation and $G = 100$, is itself significantly better than the original POEMS on all seven benchmarks: it reduces the dead space by 44–58 % on six of them and by 12 % on xerox. LA-POEMS is significantly better than the improved POEMS on five benchmarks, by 11–38 % ($\hat{A}_{12} = 0.83$–1), and better but not significantly on hp and ami49 ($\hat{A}_{12} = 0.58$ and 0.80).

Benchmark	original	improved	LA-POEMS	$\hat{A}_{12}$ of LA-POEMS vs. original	$\hat{A}_{12}$ of LA-POEMS vs. improved
apte	$0.78_{\pm 0.00}$ (0.78)	$0.78_{\pm 0.00}$ (0.78)	$0.78_{\pm 0.00}$ (0.78)	0.50	0.50
xerox	$3.23_{\pm 0.16}$ (3.16)	$\mathbf{2.85_{\pm 0.39}}$ (**2.34**)	$\mathbf{2.32_{\pm 0.02}}$ (**2.30**)	1.00	0.91
hp	$4.57_{\pm 1.23}$ (2.28)	$\mathbf{2.13_{\pm 1.46}}$ (**1.32**)	$1.62_{\pm 0.79}$ (1.32)	0.98	0.58
ami33	$3.31_{\pm 0.56}$ (2.33)	$\mathbf{1.86_{\pm 0.36}}$ (**1.37**)	$\mathbf{1.16_{\pm 0.40}}$ (**0.63**)	1.00	0.91
ami49	$3.96_{\pm 0.44}$ (3.46)	$\mathbf{2.14_{\pm 0.59}}$ (**1.65**)	$1.69_{\pm 0.34}$ (1.36)	1.00	0.80
n100	$4.97_{\pm 0.18}$ (4.69)	$\mathbf{2.41_{\pm 0.21}}$ (**2.06**)	$\mathbf{2.15_{\pm 0.17}}$ (**1.95**)	1.00	0.83
n200	$10.02_{\pm 0.76}$ (9.01)	$\mathbf{4.18_{\pm 0.29}}$ (**3.82**)	$\mathbf{2.75_{\pm 0.32}}$ (**2.47**)	1.00	1.00
n300	$11.01_{\pm 0.65}$ (10.15)	$\mathbf{5.06_{\pm 0.21}}$ (**4.74**)	$\mathbf{3.39_{\pm 0.36}}$ (**2.90**)	1.00	1.00

Table 5.11: Final comparison of the original POEMS, the improved POEMS and LA-POEMS on the tuning benchmarks (dead space [%], 60 s, seeds 101–108; mean, standard deviation and best run). Each algorithm is tested against the column to its left; the last columns give the probability that a run of LA-POEMS ends with less dead space than a run of the other algorithm

5.5.2 Held-Out Benchmarks

Table 5.12 shows the results on the six held-out instances, which were not used during the development of any of the algorithms. The picture is the same as on the tuning benchmarks. On the four instances with 100 and 200 modules, LA-POEMS is significantly better than the original POEMS in every comparison of runs ($\hat{A}_{12} = 1$) and reduces the dead space by 63–73 %. On the two small instances, n30b and n50b, its mean dead space is lower by 27 and 37 % ($\hat{A}_{12} = 0.72$ and 0.88), but with five runs the differences are not significant. The improved POEMS is significantly better than the original POEMS on the four larger instances, by 54–60 %. LA-POEMS is significantly better than the improved POEMS on n30b, n100b, n200b and n200c, by 18–35 % ($\hat{A}_{12} \geq 0.92$). On n100c, it is better by 17 %, but not significantly, and on n50b the improved POEMS has the lower mean (1.70 against 1.85 %), also not significantly.

5.5.3 Convergence

Fig. 5.2 shows how the best floorplan improves during the runs, and Table 5.13 how long the improved POEMS and LA-POEMS need to reach the final quality of the original POEMS. The original POEMS improves steadily from its first iteration. The improved POEMS starts much faster and reaches the 60-second result of the original algorithm within 0.2–4 seconds on all benchmarks except hp (14 seconds). LA-POEMS keeps its best floorplan close to the prototype during the first part of the run: at the high initial temperature, simulated annealing accepts many deteriorations, and the current floorplan is mostly worse

Benchmark	original	improved	LA-POEMS	$\hat{A}_{12}$ of LA-POEMS vs. original	$\hat{A}_{12}$ of LA-POEMS vs. improved
n30b	$2.35_{\pm 0.65}$ (1.56)	$2.61_{\pm 0.64}$ (1.90)	$\mathbf{1.70_{\pm 0.28}}$ (**1.28**)	0.72	0.92
n50b	$2.91_{\pm 0.64}$ (1.69)	$1.70_{\pm 0.12}$ (1.57)	$1.85_{\pm 0.29}$ (1.52)	0.88	0.44
n100b	$5.23_{\pm 0.28}$ (4.97)	$\mathbf{2.40_{\pm 0.18}}$ (**2.04**)	$\mathbf{1.95_{\pm 0.12}}$ (**1.74**)	1.00	0.96
n100c	$5.75_{\pm 0.74}$ (5.04)	$\mathbf{2.51_{\pm 0.17}}$ (**2.35**)	$2.07_{\pm 0.29}$ (1.70)	1.00	0.86
n200b	$9.71_{\pm 0.56}$ (8.76)	$\mathbf{4.04_{\pm 0.21}}$ (**3.64**)	$\mathbf{2.86_{\pm 0.11}}$ (**2.71**)	1.00	1.00
n200c	$10.27_{\pm 1.02}$ (9.03)	$\mathbf{4.15_{\pm 0.25}}$ (**3.69**)	$\mathbf{2.75_{\pm 0.26}}$ (**2.28**)	1.00	1.00

Table 5.12: Final comparison of the original POEMS, the improved POEMS and LA-POEMS on the held-out benchmarks (dead space [%], 60 s, seeds 101–105; mean, standard deviation and best run). Each algorithm is tested against the column to its left; the last columns give the probability that a run of LA-POEMS ends with less dead space than a run of the other algorithm

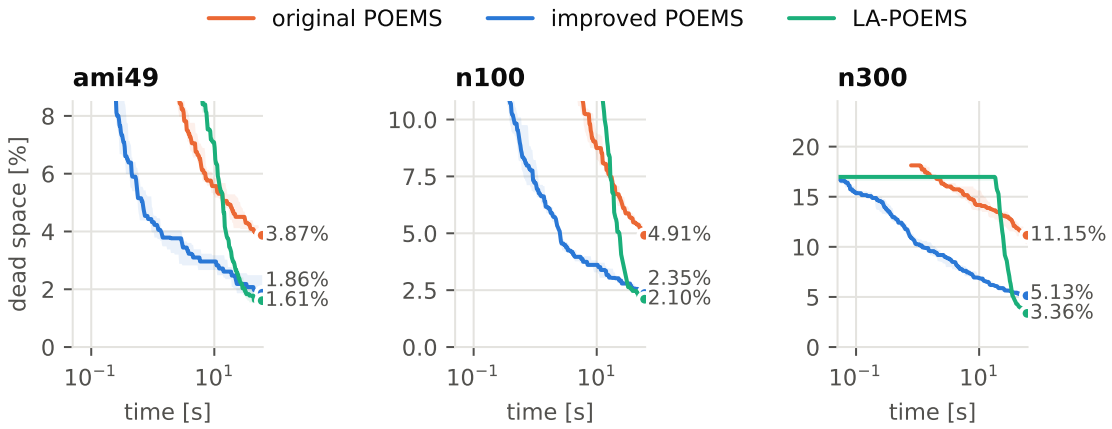


Figure 5.2: Convergence of the three algorithms on ami49, n100 and n300: median dead space of the best floorplan found so far over the 8 runs (seeds 101–108), with the interquartile range shaded; logarithmic time axis

than the prototype. As the temperature decreases, the best floorplan improves quickly. On ami33, ami49 and the GSRC instances, LA-POEMS reaches the target after 10–25 seconds and overtakes the improved POEMS after 15–34 seconds; on apte, xerox and hp, it reaches the target within half a second. LA-POEMS with simulated annealing is therefore not an anytime algorithm: its cooling schedule is fitted to the time budget, which has to be known in advance, and a run stopped early returns a poor floorplan. When a run may be stopped at any time, the improved POEMS or late acceptance (Section 5.4), which improve the best floorplan continuously, are the better choice.

Benchmark	target [%]	improved POEMS	LA-POEMS
apte	0.78	0.2	0.0
xerox	3.16	0.5	0.3
hp	4.58	13.7	0.1
ami33	3.47	4.0	9.6
ami49	3.87	2.2	15.4
n100	4.91	2.6	23.6
n200	10.02	1.4	21.7
n300	11.15	0.7	24.5

Table 5.13: Median time [s] in which the improved POEMS and LA-POEMS first reach the median final dead space of the original POEMS (the target, 60 s runs, seeds 101–108; – = not reached by half of the runs)

5.5.4 Long Runs and Published Results

Table 5.14 puts the results into the context of the original edition and of published results. The comparison with published results is not controlled (Section 5.1): they were obtained on much slower computers, and their times are given only for orientation. Within 60 seconds, LA-POEMS finds on average less dead space than the longest configuration of the original edition ($I = 20\,000$ iterations), which ran for 12 to 153 minutes on a computer about 1.7 times slower, on every benchmark except apte, where all algorithms find the same floorplan. Its mean dead space is also lower than that of the hard-block packings of CompaSS [3], which are computed within seconds, on every benchmark except apte. The B*-Tree with simulated annealing [4] reports the areas of its floorplans with two decimals. Computed from them, its dead space on apte and hp equals that of the best floorplans of LA-POEMS within the rounding, and on xerox, ami33 and ami49, LA-POEMS finds smaller floorplans on average. Longer runs pay off on the large instances: in five minutes, LA-POEMS reduces the mean dead space on n100, n200 and n300 by another 26–31 % compared with the one-minute runs with the same seeds, to 1.66, 2.03 and 2.38 % (significantly). On ami33 and ami49, the differences between the one- and the five-minute runs are not significant; on ami33, the mean of the five-minute runs is even higher (1.20 against 1.02 %), although their best run reaches 0.84 %. The original edition concluded that POEMS outperforms CompaSS on all benchmarks, which its own results did not support for hp (3.88 % in its long run against 2.28 %). For LA-POEMS, the claim holds on average over one-minute runs on every benchmark except apte, where both find the same floorplan. Fig. 5.3 shows the best floorplans found by LA-POEMS.

Answer to RQ4. With the same time budget of one minute and new seeds, both new algorithms are much better than the original POEMS. The improved POEMS reduces its mean dead space by 44–58 % on the tuning instances with 11 or more modules, and LA-POEMS by 57–73 % on those with 33 or more modules. LA-POEMS is significantly

	LA-POEMS		original edition		CompaSS [3]		B*-Tree SA [4]	
Benchmark	60 s	300 s	[%]	time [s]	[%]	time [s]	[%]	time [s]
apte	0.78 (0.78)	–	0.78	771	0.78	0.00	0.77	7
xerox	2.32 (2.30)	–	2.77	723	3.91	0.26	2.48	25
hp	1.62 (1.32)	–	3.13	800	2.28	0.81	1.35	55
ami33	1.16 (0.63)	1.20 (0.84)	2.29	1277	4.94	6.19	9.82	3417
ami49	1.69 (1.36)	1.51 (1.07)	2.56	1699	4.87	6.34	3.82	4752
n100	2.15 (1.95)	1.66 (1.49)	2.93	3145	7.32	4.17	–	–
n200	2.75 (2.47)	2.03 (1.61)	4.32	6067	6.48	10.43	–	–
n300	3.39 (2.90)	2.38 (2.29)	4.73	9200	6.80	6.87	–	–

Table 5.14: LA-POEMS compared with the original edition and with published results: dead space [%] of LA-POEMS (mean and best of 8 runs of 60 s, seeds 101–108, and of 5 runs of 300 s, seeds 101–105), the mean of 10 runs of the longest configuration of the original edition ($I = 20\,000$, on a 2 GHz computer), CompaSS [27, 28] (2.8 GHz Pentium 4) and the B*-Tree with simulated annealing [4] (computed from the areas reported with two decimals, 200 MHz Sun Ultra-I); times as reported

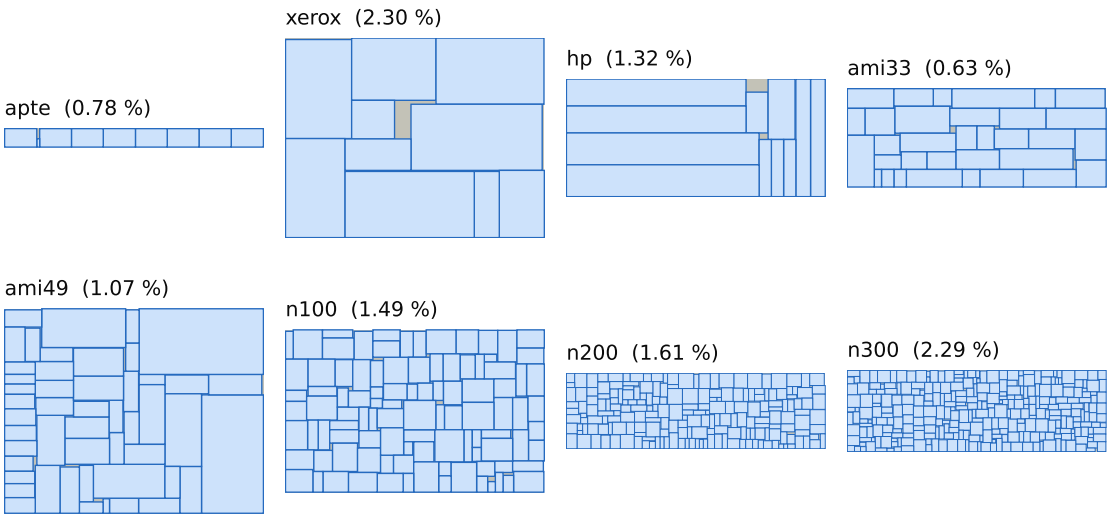


Figure 5.3: The best floorplans found by LA-POEMS in the final comparison and in the long runs, with the dead space in gray (the longer side is drawn horizontally). The objective does not constrain the aspect ratio, so some floorplans, such as that of apte, are long strips

better than the improved POEMS on five of the seven non-trivial tuning benchmarks and on four of the six held-out instances, and it is not significantly worse on any instance. The advantage has a price: with simulated annealing, the best floorplan improves mainly in the second half of the run, so the time budget must be known in advance. Five-minute runs reduce the dead space on the three largest instances by another 26–31 %. Within one minute, LA-POEMS finds on average less dead space than the original edition after up to 2.5 hours and than the published results of CompaSS on every benchmark except apte, where all algorithms find the same floorplan; compared with the B*-Tree with simulated annealing, its best floorplans are equal on apte and hp and smaller on the other three MCNC benchmarks.

5.6 Threats to Validity

Tuning on the same benchmarks. The configurations of the improved POEMS and of LA-POEMS were chosen on the same eight benchmarks on which they are compared. Two measures reduce the resulting optimistic bias: the final comparison uses seeds that were not used for tuning, and the held-out instances were not used at all before the final comparison.

Implementation and algorithm. The original POEMS runs in its original, slower implementation, so its comparison with the two new algorithms mixes the effects of the implementation and of the algorithm. The improved POEMS and LA-POEMS share the implementation; their comparison, and that of Table 5.10, isolates the algorithm.

Time measurement. The runs share a machine with other runs, so their speed varies by a few percent; runs that are compared were always started in the same configuration of parallel runs. The time limits include the just-in-time compilation, which matters only for the shortest runs.

Sample size and multiple comparisons. Five to eight seeds per configuration detect only large differences, and non-significant differences in the screening tables should not be read as evidence of equal performance. Conversely, the screening tables contain several hundred tests without a correction for multiple comparisons; at the level 0.05, about one test in twenty is significant by chance, so a single significant cell is weak evidence. The conclusions of this chapter therefore rely on consistent patterns across benchmarks and on the final comparison with new seeds.

Comparison with the literature. The results of other algorithms are taken from their publications, which used other machines, other time budgets and in some cases other definitions of the measured quantity. They indicate the order of magnitude of the results; they are not a controlled comparison.

Chapter **6**

Conclusions

6.1 Summary

The thesis studied the minimisation of the area of the enclosing rectangle of rotatable rectangular modules, represented by B*-Trees and optimised by POEMS, an iterative algorithm that evolves sequences of tree modifications by a genetic algorithm. The revised edition re-examined the original work and extended it in four ways.

First, the implementation was profiled and made 7.5 to 29 times faster without changing its results. The original placement was superlinear, and most of the remaining time went to temporary objects and to the formatting of discarded messages. Within equal time, the faster implementation alone finds significantly better floorplans on all larger benchmarks (RQ1).

Second, seven defects were found and corrected; the most important one prevented rotating a module by itself. Instrumentation showed that the genetic algorithm of POEMS wastes most of its evaluations: almost half of its children duplicate individuals of the population, the step applied in an iteration is usually found among the first individuals, and most iterations only move along a plateau of the objective. Of the modifications of the genetic algorithm, only shorter iterations helped: with 100 instead of 1000 generations, POEMS makes 5.6 times more iterations and is significantly better on four of seven benchmarks (RQ2).

Third, late acceptance POEMS keeps the representation, the actions and the action sequences of POEMS, but applies random sequences immediately and accepts them by late acceptance or simulated annealing. On the same implementation and with the same time, it is significantly better than POEMS on most instances and worse on none, so the

genetic algorithm is not necessary for finding good steps. The acceptance criterion is the most important design choice; the final configuration uses simulated annealing, steps of at most two actions and local partners (RQ3).

Fourth, the algorithms were compared with equal time budgets, new seeds, held-out instances and non-parametric statistics. Within one minute, LA-POEMS reduced the dead space of the original POEMS by 57–73 % on the instances with 33 to 300 modules and was significantly better than the improved POEMS on five of seven tuning benchmarks and on four of six held-out instances. Its results are as good as or better than those of the original edition after up to 2.5 hours and the published results of CompaSS, and its best floorplans are at least as good as those of the B*-Tree with simulated annealing. The price of simulated annealing is that the time budget must be known in advance (RQ4).

6.2 Lessons

- *Measure the implementation before the algorithm.* Implementation overhead changed the results of POEMS more than most algorithmic modifications, so conclusions about an algorithm drawn from a slow implementation can be misleading.
- *Look at what the evolutionary algorithm does, not only at its result.* Simple counters (duplicates, generation of the best individual, outcome of iterations) revealed that most of the work of the genetic algorithm did not influence the result.
- *Compare with a simple baseline on the same representation.* Applying the same actions with a simple acceptance criterion turned out to be a stronger algorithm than the original method; without this baseline, the merit of the genetic algorithm cannot be judged.
- *Use equal budgets, several seeds and statistical tests.* Several differences that looked convincing in single runs disappeared over six or eight seeds, and some modifications helped on one group of instances and hurt on another.

6.3 Future Work

- **Fixed-outline floorplanning and wire length.** Practical floorplanners place the modules into a given outline and minimise the wire length [1, 5]. Both objectives can be added to LA-POEMS through the cost of a candidate, and the early termination of the placement still applies to the outline constraint.

- **Incremental evaluation.** After a local modification, only the modules placed after the first changed node in the depth-first order move. Keeping the contour at checkpoints of the traversal could avoid repeating the placement of the unchanged prefix.
- **Parallel search.** Independent runs of LA-POEMS, or runs that periodically exchange their best solutions, would use all cores of a modern computer.
- **Evolution of the steps revisited.** The genetic algorithm of POEMS could be used as an occasional escape mechanism of LA-POEMS from deep local optima, or it could evolve the parameters of the distribution of the steps instead of the steps themselves.
- **Soft and rectilinear modules**, which B*-Trees support [4], as suggested in the original edition.

Appendix A

Using the Program

A.1 Building

The program is built by Maven, which also runs the unit tests and creates a runnable archive:

```
mvn package
java -jar target/program.jar
```

A.2 Command Line

The program solves every benchmark file found in the input directory and writes the results into a subdirectory of the output directory for each run. The nine mandatory arguments of the original edition are kept (Table A.1); they can be followed by the optional arguments of Table A.2. Without the optional arguments, the program runs POEMS with the corrections of Section 4.5; the modifications evaluated in Chapter 5 are switched on by the system properties of Table A.3.

```
program.jar IN OUT VERBOSE R I G N S BESTFIT [OPTIONS]
program.jar benchmark/ benchmark/result/ false 1 1000 500 3 50 true
program.jar benchmark/ benchmark/result/ false 5 0 0 2 0 true \
    --algorithm=lapoems --time=60
```

Name	Description	Example
`IN`	input directory	benchmark/
`OUT`	output directory	benchmark/result/
`VERBOSE`	verbose mode	false
`R`	number of runs for each benchmark	1
`I`	number of iterations	1000
`G`	number of generations	500
`N`	niche count (length of action sequences; LA-POEMS: maximal number of actions in a step)	3
`S`	niche size (number of individuals in a niche)	50
`BESTFIT`	best-fit prototype (otherwise complete tree)	true

Table A.1: Mandatory command line arguments

Option	Description
`-algorithm=poems|lapoems`	the solver: POEMS (default) or late acceptance POEMS
`-time=SECONDS`	time limit of each run; without it, LA-POEMS makes $I \cdot G$ steps
`-seed=NUMBER`	seed of the random number generator
`-history=LENGTH`	LA-POEMS: late acceptance with this history length (0 = automatic) instead of simulated annealing

Table A.2: Optional command line arguments

```
11
1036.00 462.00
378.00 700.00
980.00 210.00
980.00 210.00
980.00 210.00
3304.00 546.00
3304.00 546.00
2016.00 252.00
3080.00 462.00
2016.00 252.00
3080.00 462.00
```

Figure A.1: The MCNC hp benchmark file

For each run, the output directory contains the final floorplan as an SVG image (`floorplan.svg`, and `floorplan_s.svg` with the tree drawn over it), the tree as a DOT graph, its evaluation, the prototype, and the floorplans of all improvements of POEMS, which can be assembled into an animation.

A.3 Benchmark Files

A benchmark file is a plain text file that starts with the number of modules, followed by the width and the height of every module (the format of the benchmarks distributed with the floorplanning tools of the University of Michigan [26]). The dimensions may be written as real numbers, but their fractional parts must be zero. The whole MCNC hp benchmark is shown in Fig. A.1.

A.4 Reproducing the Experiments

The benchmark runner solves one benchmark with a given seed and time limit (in milliseconds) and prints one line: the benchmark, the seed, the dead space in percent, the unused, total and used area, the width and height, the number of evaluations and iterations, the time, and the result of the validation of the floorplan.

Property	Default	Meaning
`poems.algorithm`	`poems`	`poems` or `lapoems`
`poems.fast`	true	POEMS: array evaluation (identical results, faster)
`poems.dedupe`	false	POEMS: discard children whose sequence is in the population
`poems.patience`	0	POEMS: end an iteration after P generations without improvement (0 = never)
`poems.first`	false	POEMS: end an iteration when the prototype is improved
`poems.lahc`	0	POEMS: late acceptance of prototypes with a history of L iterations (0 = off)
`poems.move`	false	POEMS: use the move node action
`poems.t0`, `poems.t1`	10^{-2}, 10^{-5}	LA-POEMS: initial and final temperature of simulated annealing, relative to the area of the modules ($T_0 = 0$: late acceptance)
`poems.ratio`	20000	LA-POEMS: automatic history length = expected steps / C
`poems.restart`	0	LA-POEMS: restart after $R \cdot n$ steps without improvement (0 = never)
`poems.local`	0.5	LA-POEMS: probability of a local second module
`poems.weights`	uniform	LA-POEMS: relative probabilities of the seven action types
`poems.adaptive`	false	LA-POEMS: adaptive selection of the action types
`poems.trace`	–	benchmark runner: write the convergence trace into a file or directory
`poems.placement`	–	benchmark runner: write the final floorplan into a directory
`poems.evaluations`	0	benchmark runner, LA-POEMS: stop after this number of steps (0 = time limit only)

Table A.3: System properties of the experimental settings

```
java -cp target/program.jar program.Benchmark \
    benchmark/n100.txt SEED TIME_MS I G N S
java -cp target/program.jar -Dpoems.algorithm=lapoems \
    program.Benchmark benchmark/n100.txt SEED TIME_MS 0 0 N L
```

For LA-POEMS, `N` is the maximal number of actions in a step and `L` the late acceptance history length, which is used only with `-Dpoems.t0=0` (0 = automatic).

The settings of the improved POEMS and of LA-POEMS that were evaluated in Chapter 5 are switched by Java system properties, listed in Table A.3. The directory `experiments` of the repository [29] contains the scripts that ran all experiments of Chapter 5, the raw results of every run, and the scripts that create the tables and figures of this thesis from them.

Bibliography

[1] S. N. Adya, I. L. Markov: *Fixed-Outline Floorplanning: Enabling Hierarchical Design*, IEEE Transactions on Very Large Scale Integration (VLSI) Systems, vol. 11, no. 6, pp. 1120–1135, 2003.

[2] E. K. Burke, Y. Bykov: *The Late Acceptance Hill-Climbing Heuristic*, European Journal of Operational Research, vol. 258, no. 1, pp. 70–78, 2017.

[3] H. H. Chan, I. L. Markov: *Practical Slicing and Non-Slicing Block-Packing without Simulated Annealing*, Proceedings of the Great Lakes Symposium on VLSI (GLSVLSI), pp. 282–287, 2004.

[4] Y.-C. Chang, Y.-W. Chang, G.-M. Wu, S.-W. Wu: *B*-Trees: A New Representation for Non-Slicing Floorplans*, Proceedings of the 37th Design Automation Conference, pp. 458–463, 2000.

[5] T.-C. Chen, Y.-W. Chang: *Modern Floorplanning Based on B*-Tree and Fast Simulated Annealing*, IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, vol. 25, no. 4, pp. 637–650, 2006.

[6] A. E. Eiben, J. E. Smith: *Introduction to Evolutionary Computing*, 2nd edition, Springer, 2015.

[7] R. J. Fowler, M. S. Paterson, S. L. Tanimoto: *Optimal Packing and Covering in the Plane are NP-Complete*, Information Processing Letters, vol. 12, no. 3, pp. 133–137, 1981.

[8] M. R. Garey, D. S. Johnson: *Computers and Intractability: A Guide to the Theory of NP-Completeness*, W. H. Freeman, 1979.

[9] D. E. Goldberg, K. Deb: *A Comparative Analysis of Selection Schemes Used in Genetic Algorithms*, Foundations of Genetic Algorithms, pp. 69–93, Morgan Kaufmann, 1991.

[10] P.-N. Guo, T. Takahashi, C.-K. Cheng, T. Yoshimura: *Floorplanning Using a Tree Representation*, IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, vol. 20, no. 2, pp. 281–289, 2001.

[11] J. H. Holland: *Adaptation in Natural and Artificial Systems*, University of Michigan Press, 1975.

[12] X. Hong, G. Huang, Y. Cai, J. Gu, S. Dong, C.-K. Cheng, J. Gu: *Corner Block List: An Effective and Efficient Topological Representation of Non-Slicing Floorplan*, Proceedings of the International Conference on Computer-Aided Design, pp. 8–12, 2000.

[13] V. Hordějčuk: *Optimisation of Rectangular Shapes Placement by Means of Evolutionary Algorithms*, bachelor's thesis, Czech Technical University in Prague, 2010.

[14] S. Kirkpatrick, C. D. Gelatt, M. P. Vecchi: *Optimization by Simulated Annealing*, Science, vol. 220, no. 4598, pp. 671–680, 1983.

[15] J. Kubalík, J. Faigl: *Iterative Prototype Optimisation with Evolved Improvement Steps*, Genetic Programming (EuroGP 2006), Lecture Notes in Computer Science, vol. 3905, pp. 154–165, Springer, 2006.

[16] M. Lai, D. F. Wong: *Slicing Tree Is a Complete Floorplan Representation*, Proceedings of Design, Automation and Test in Europe (DATE), pp. 228–232, 2001.

[17] J. Y.-T. Leung, T. W. Tam, C. S. Wong, G. H. Young, F. Y. L. Chin: *Packing Squares into a Square*, Journal of Parallel and Distributed Computing, vol. 10, no. 3, pp. 271–275, 1990.

[18] J.-M. Lin, Y.-W. Chang: *TCG-S: Orthogonal Coupling of P*-Admissible Representations for General Floorplans*, IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, vol. 23, no. 6, pp. 968–980, 2004.

[19] J.-M. Lin, Y.-W. Chang: *TCG: A Transitive Closure Graph-Based Representation for General Floorplans*, IEEE Transactions on Very Large Scale Integration (VLSI) Systems, vol. 13, no. 2, pp. 288–292, 2005.

[20] H. B. Mann, D. R. Whitney: *On a Test of Whether One of Two Random Variables Is Stochastically Larger than the Other*, The Annals of Mathematical Statistics, vol. 18, no. 1, pp. 50–60, 1947.

[21] H. Murata, K. Fujiyoshi, S. Nakatake, Y. Kajitani: *VLSI Module Placement Based on Rectangle-Packing by the Sequence-Pair*, IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, vol. 15, no. 12, pp. 1518–1524, 1996.

[22] S. Nakatake, K. Fujiyoshi, H. Murata, Y. Kajitani: *Module Placement on BSG-Structure and IC Layout Applications*, Proceedings of the International Conference on Computer-Aided Design, pp. 484–491, 1996.

[23] M. Tang, X. Yao: *A Memetic Algorithm for VLSI Floorplanning*, IEEE Transactions on Systems, Man, and Cybernetics, Part B, vol. 37, no. 1, pp. 62–69, 2007.

[24] A. Vargha, H. D. Delaney: *A Critique and Improvement of the CL Common Language Effect Size Statistics of McGraw and Wong*, Journal of Educational and Behavioral Statistics, vol. 25, no. 2, pp. 101–132, 2000.

[25] D. F. Wong, C. L. Liu: *A New Algorithm for Floorplan Design*, Proceedings of the 23rd Design Automation Conference, pp. 101–107, 1986.

[26] Floorplanning benchmarks and tools of the University of Michigan, `http://vlsicad.eecs.umich.edu/BK/FPUtils/`

[27] CompaSS results for the hard-block MCNC benchmarks, `http://vlsicad.eecs.umich.edu/BK/CompaSS/results/mcnc_txt.html` (heuristic packings) and `http://vlsicad.eecs.umich.edu/BK/CompaSS/results/mcnc_opt.html` (optimal slicing packings), accessed October 2026.

[28] CompaSS results for the hard-block GSRC benchmarks, `http://vlsicad.eecs.umich.edu/BK/CompaSS/results/gsrc_txt.html`, accessed October 2026.

[29] Project repository, `https://github.com/voho/vh-master-thesis`